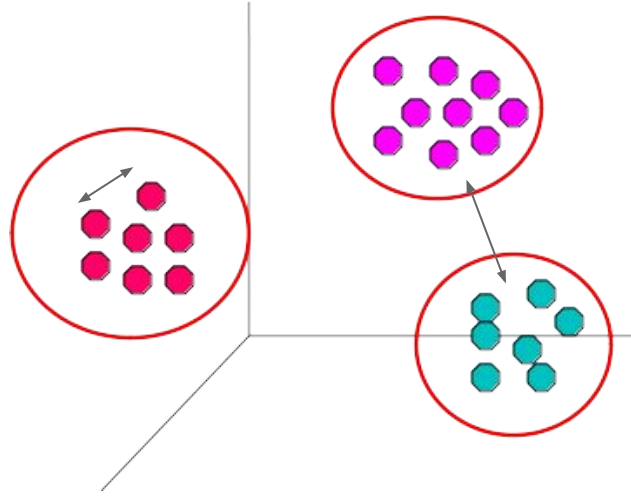


Clustering (parte 1/3)

A dark blue, abstract shape that starts as a thin line on the left and expands into a large, solid triangular area that fills the bottom half of the slide.

El problema: Agrupar objetos en distintos grupos de acuerdo a sus similitudes

Minimizar
distancias
intra grupos

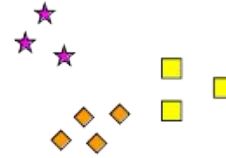
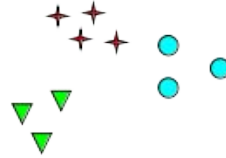


Maximizar
distancias
entre grupos

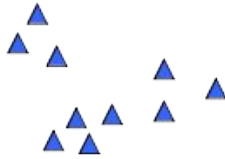
El problema: Agrupar objetos en distintos grupos de acuerdo a sus similitudes



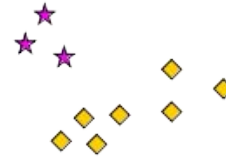
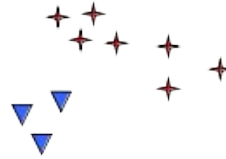
¿Cuántos clusters?



¿Seis?



¿Dos?



¿Cuatro?

Hoja de ruta

1. Algoritmos
2. Medidas de similaridad/distancia y Validación de Clusterings
3. Aplicaciones

Distintos tipos de Clustering ([Tan, Steinbach & Kumar "Introduction to Data Mining"](#))

- Clustering por prototipo (k-means / PAM o k-medoids)
- Clustering jerárquico
- Clustering por densidad (DBSCAN)
- Clustering difuso

Librerías de Python para cálculos principalmente [scikit-learn](#) y para graficar hay muchas herramientas en [yellowbrick](#). Recientemente, también apareció [clusteval](#). Por ejemplo,

```
>>> from yellowbrick.cluster import SilhouetteVisualizer
>>> from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
>>> from sklearn_extra.cluster import KMedoids
>>> import skfuzz
```

Distintos tipos de Clustering ([Tan, Steinbach & Kumar "Introduction to Data Mining"](#))

- **Particiones predeterminadas** (vía número de clusters fijo) vs. **Jerárquico** (anidado)
- **Exclusivo** (cada punto pertenece a un cluster) vs. **Difuso** (todos los puntos pertenecen a todas los clusters con cierta probabilidad)
- **Completo** (todos los puntos se asignan a algún cluster) vs. **Parcial** (hay puntos que no se asignan)

Clustering por prototipos (k-means)

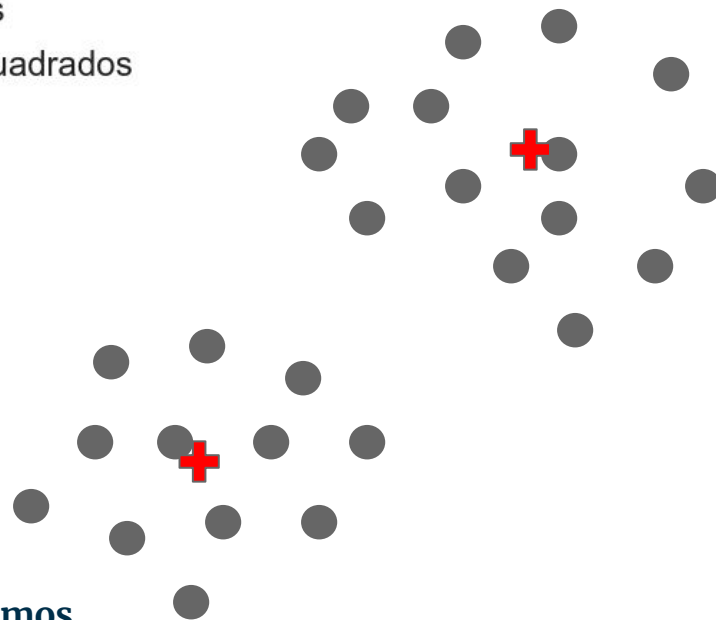
Dado un conjunto de observaciones ($\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$), donde cada observación es un vector real de d dimensiones, k-medias construye una partición de las observaciones en k conjuntos ($k \leq n$) a fin de minimizar la suma de los cuadrados dentro de cada grupo (WCSS): $\mathbf{S} = \{S_1, S_2, \dots, S_k\}$

$$\arg \min_{\mathbf{S}} \sum_{i=1}^k \sum_{\mathbf{x}_j \in S_i} \|\mathbf{x}_j - \boldsymbol{\mu}_i\|^2$$

donde $\boldsymbol{\mu}_i$ es la media de puntos en S_i .

Este problema tiene un **óptimo global** al que se puede llegar en un número finito de pasos, pero minimizar esta función objetivo es computacionalmente “difícil” (NP-hard).

Al rescate: hay heurísticas que pueden llevarme **rápidamente a óptimos locales**



Clustering por prototipos (k-means, algoritmo de Lloyd)

► Seleccionar K.

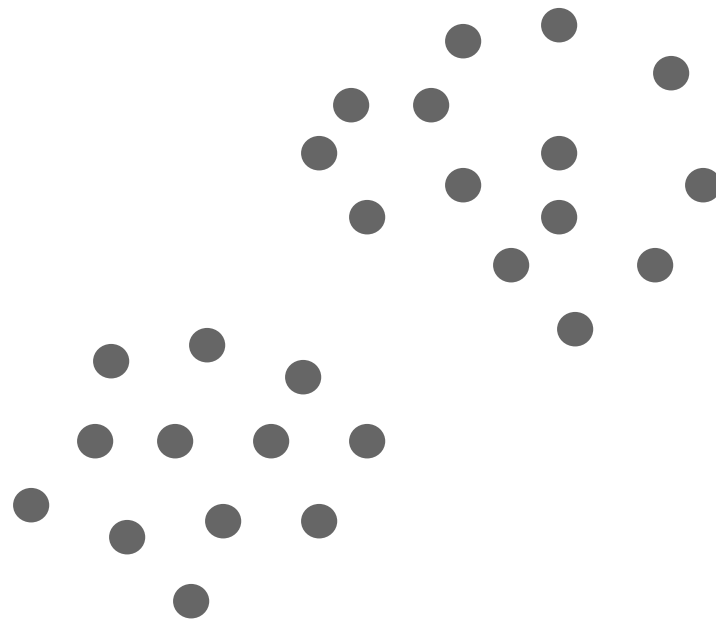
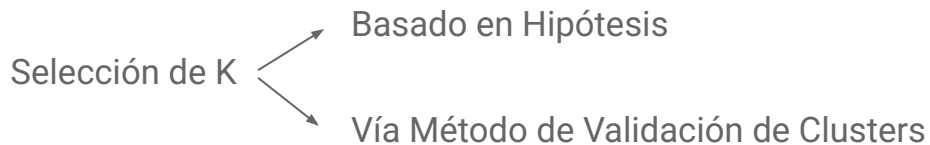
Seleccionar K puntos como centroides iniciales.

Repetir:

Asignar cada punto a uno de los K clusters.

Recomputar los centroides de cada clusters.

Hasta que: los centroides no cambien



Clustering por prototipos (k-means)

► Seleccionar $K = 2$.

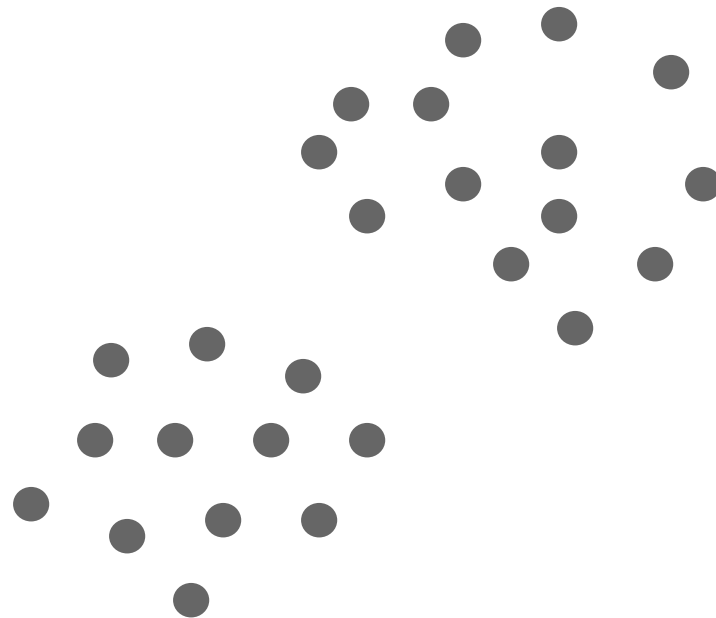
Seleccionar K puntos como centroides iniciales.

Repetir:

Asignar cada punto a uno de los K clusters.

Recomputar los centroides de cada clusters.

Hasta que: los centroides no cambien



Clustering por prototipos (k-means)

Seleccionar K .

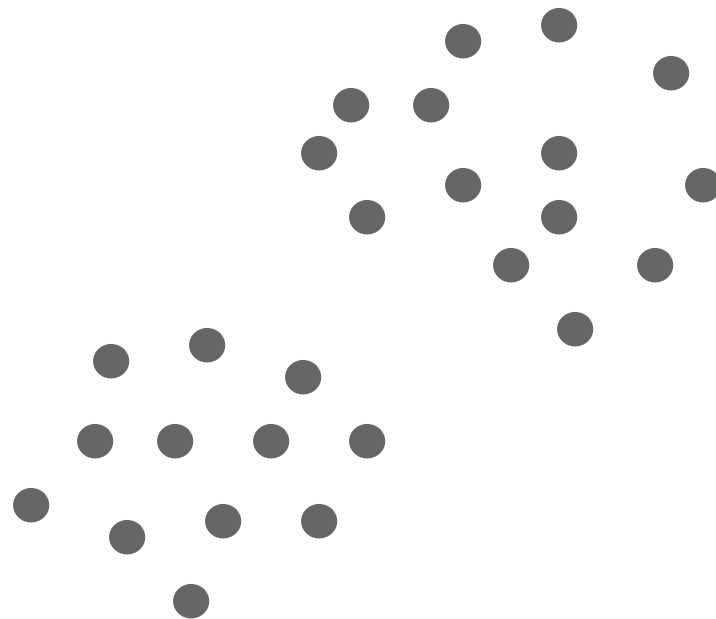
► Seleccionar K puntos como centroides iniciales.

Repetir:

Asignar cada punto a uno de los K clusters.

Recomputar los centroides de cada clusters.

Hasta que: los centroides no cambien



Clustering por prototipos (k-means)

Seleccionar K .

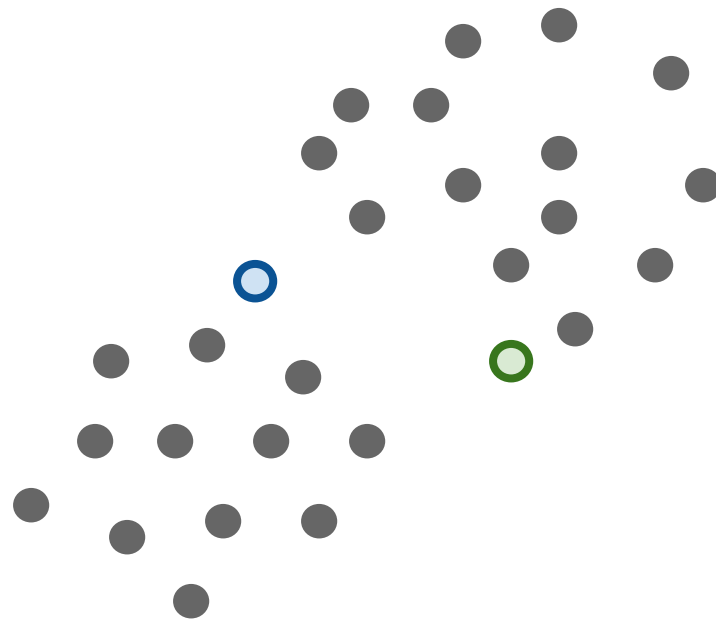
► Seleccionar K puntos como centroides iniciales.

Repetir:

Asignar cada punto a uno de los K clusters.

Recomputar los centroides de cada clusters.

Hasta que: los centroides no cambien



Clustering por prototipos (k-means)

Seleccionar K .

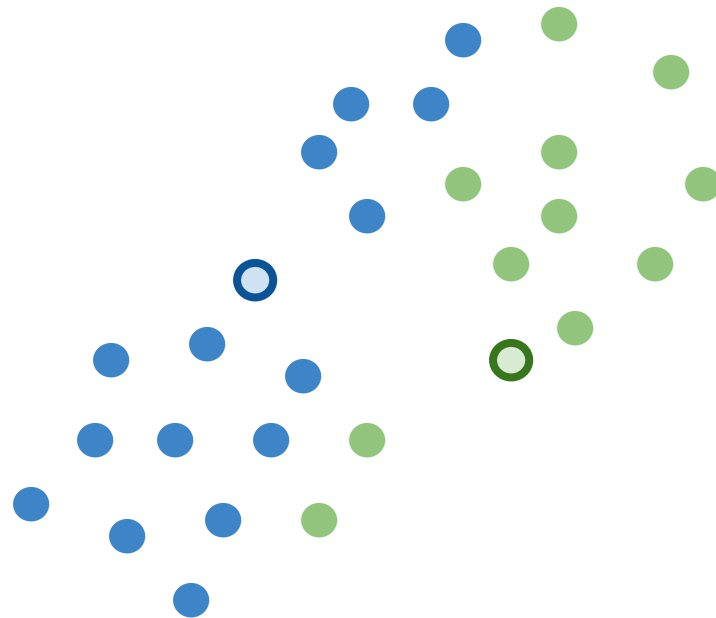
Seleccionar K puntos como centroides iniciales.

Repetir:

▶ Asignar cada punto a uno de los K clusters.

Recomputar los centroides de cada clusters.

Hasta que: los centroides no cambien



Clustering por prototipos (k-means)

Seleccionar K .

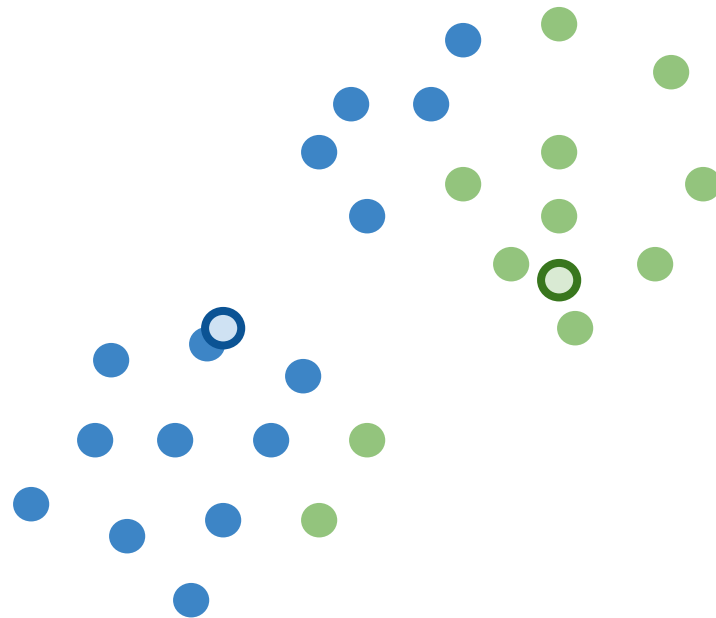
Seleccionar K puntos como centroides iniciales.

Repetir:

Asignar cada punto a uno de los K clusters.

▶ Recomputar los centroides de cada clusters.

Hasta que: los centroides no cambien



Clustering por prototipos (k-means)

Seleccionar K .

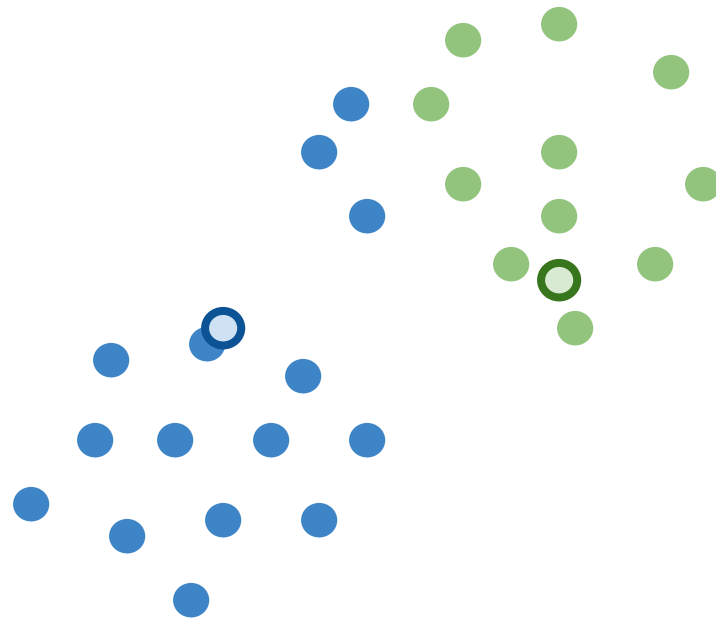
Seleccionar K puntos como centroides iniciales.

Repetir:

▶ Asignar cada punto a uno de los K clusters.

Recomputar los centroides de cada clusters.

Hasta que: los centroides no cambien



Clustering por prototipos (k-means)

Seleccionar K .

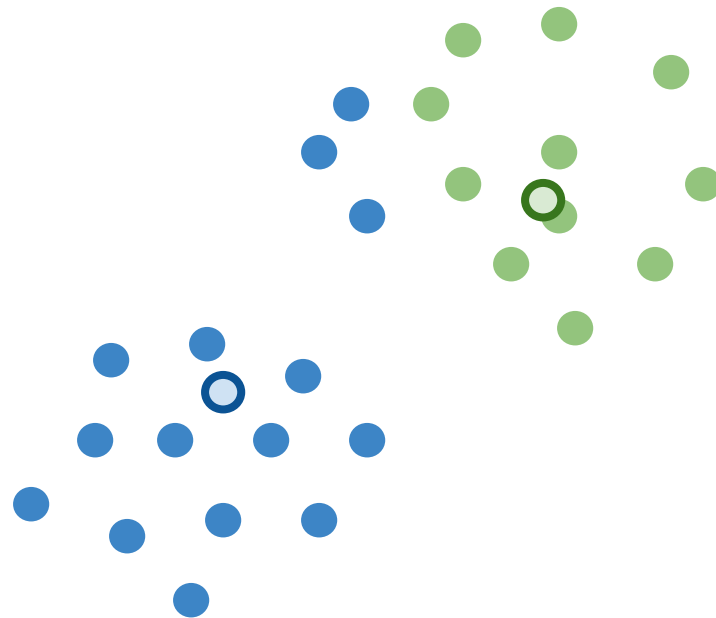
Seleccionar K puntos como centroides iniciales.

Repetir:

Asignar cada punto a uno de los K clusters.

▶ Recomputar los centroides de cada clusters.

Hasta que: los centroides no cambien



Clustering por prototipos (k-means)

Seleccionar K .

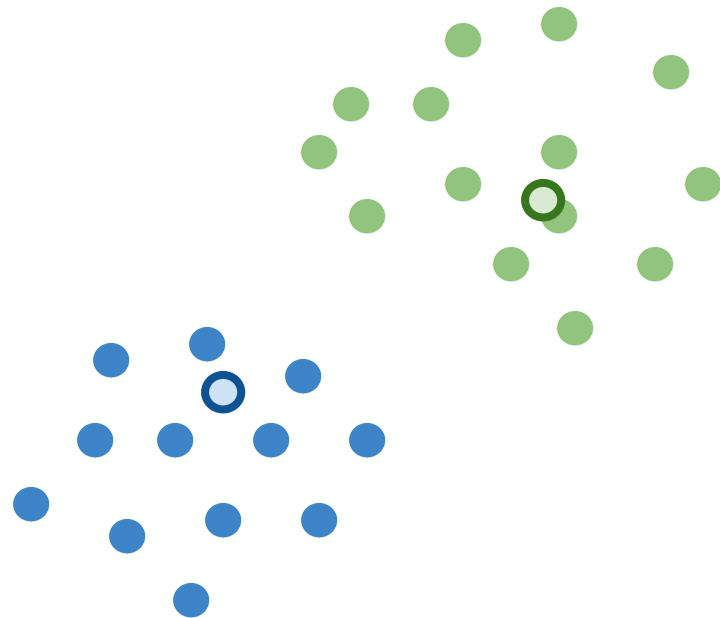
Seleccionar K puntos como centroides iniciales.

Repetir:

▶ Asignar cada punto a uno de los K clusters.

Recomputar los centroides de cada clusters.

Hasta que: los centroides no cambien



Clustering por prototipos (k-means)

Seleccionar K .

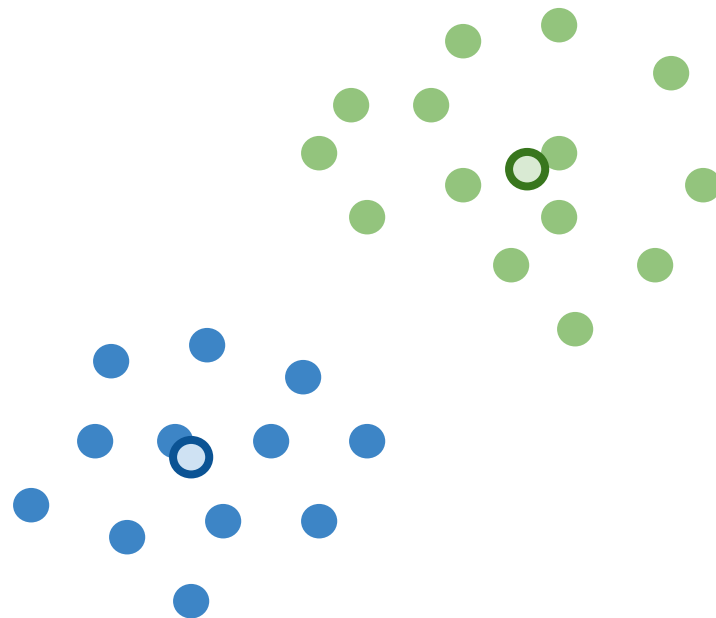
Seleccionar K puntos como centroides iniciales.

Repetir:

Asignar cada punto a uno de los K clusters.

▶ Recomputar los centroides de cada clusters.

Hasta que: los centroides no cambien



Clustering por prototipos (k-means)

Seleccionar K .

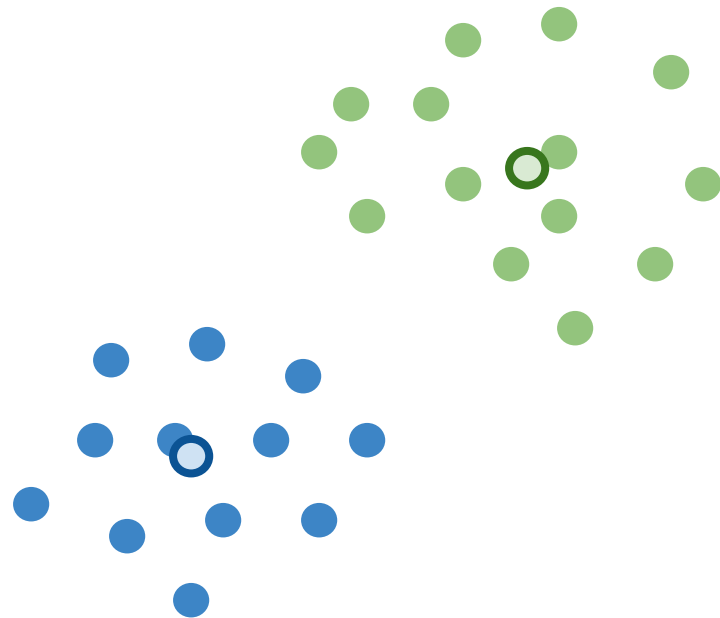
Seleccionar K puntos como centroides iniciales.

Repetir:

▶ Asignar cada punto a uno de los K clusters.

Recomputar los centroides de cada clusters.

Hasta que: los centroides no cambien



Clustering por prototipos (k-means)

Seleccionar K .

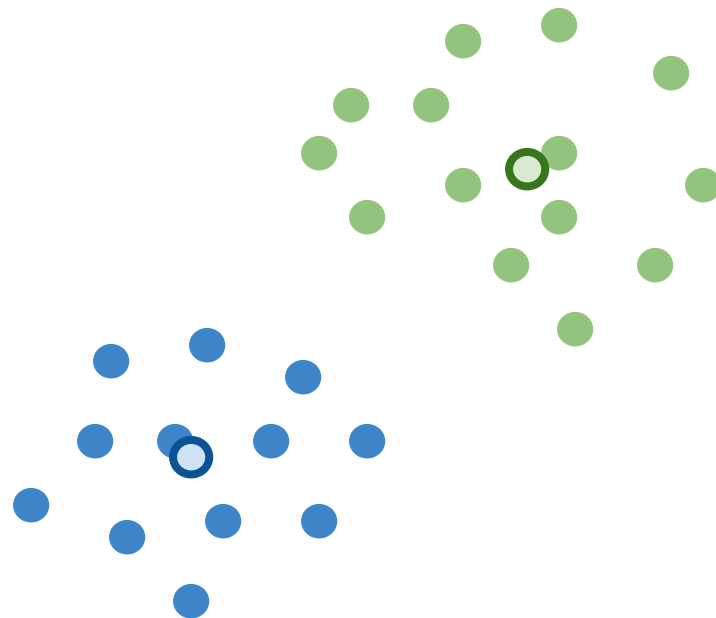
Seleccionar K puntos como centroides iniciales.

Repetir:

Asignar cada punto a uno de los K clusters.

▶ Recomputar los centroides de cada clusters.

Hasta que: los centroides no cambien



Clustering por prototipos (k-means)

Seleccionar K .

Seleccionar K puntos como centroides iniciales.

Repetir:

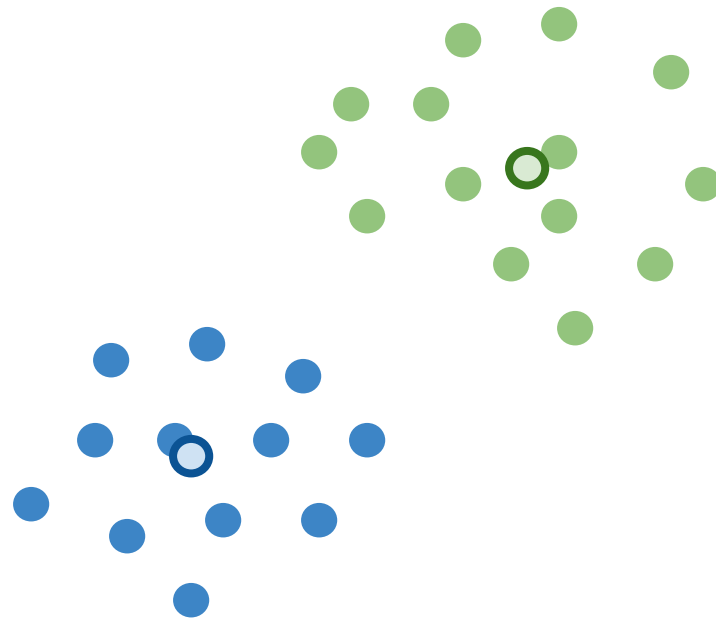
Asignar cada punto a uno de los K clusters.

Recomputar los centroides de cada clusters.

► **Hasta que:** los centroides no cambien

La partición final obtenida se puede valorar por la suma de “errores” al cuadrado (SSE):

$$SSE = \sum_{i=1}^K \sum_{x \in C_i} d^2(x - x_{c_i}) = \sum_{i=1}^K \sum_{x \in C_i} \|x - x_{c_i}\|^2$$



Clustering por prototipos (k-means)

Seleccionar K .

Seleccionar K puntos como centroides iniciales.

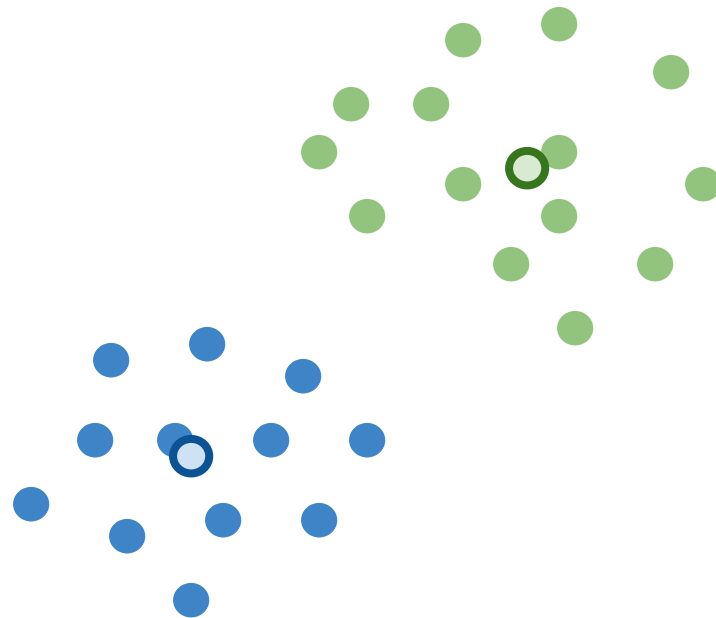
Repetir:

Asignar cada punto a uno de los K clusters.

Recomputar los centroides de cada clusters.

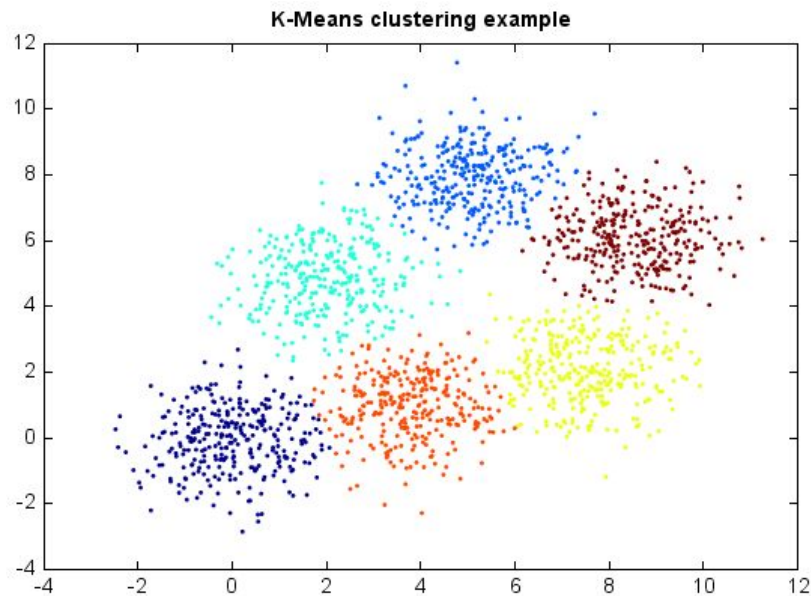
► **Hasta que:** los centroides no cambien

- Estocástico: no hay garantía de converger al óptimo global, y el resultado puede depender de los centroides iniciales. Es común ejecutarlo varias veces con diferentes condiciones iniciales.
- Eficiencia computacional: generalmente es rápido, pero para condiciones “worst-case scenario” puede llegar a tener complejidad super polinomial (muy lento). Sin embargo, estos casos son “patológicos”, y en la práctica para datasets reales el algoritmo es polinomial (muchas veces lineal).



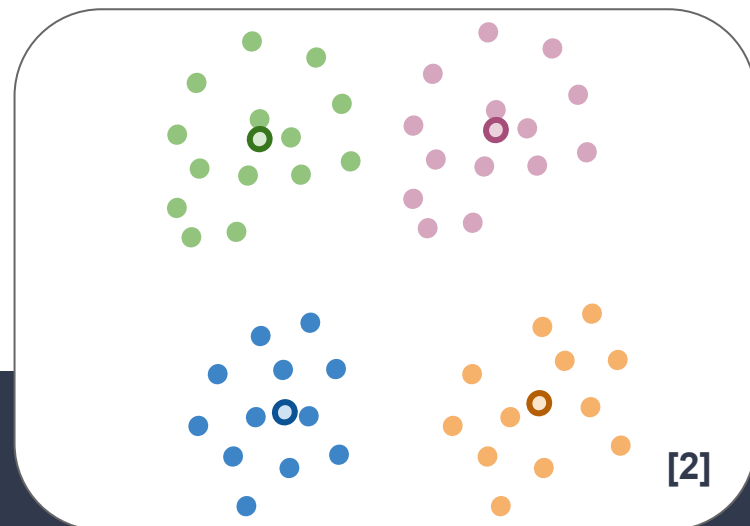
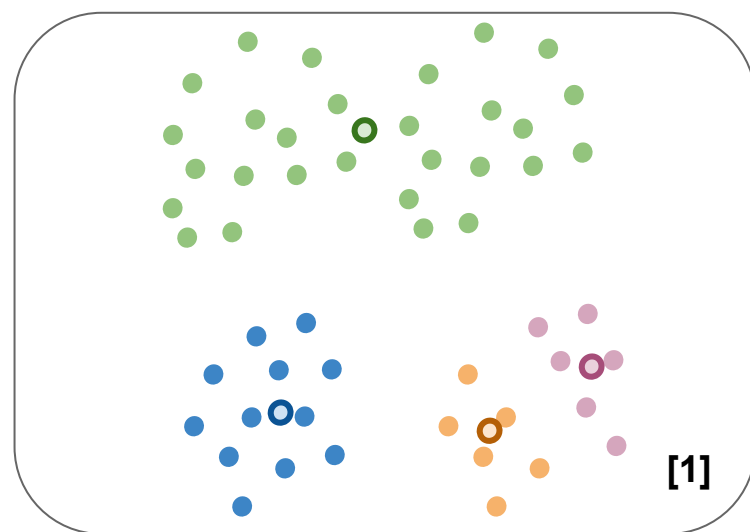
Clustering por prototipos (k-means)

Una success story de k-means



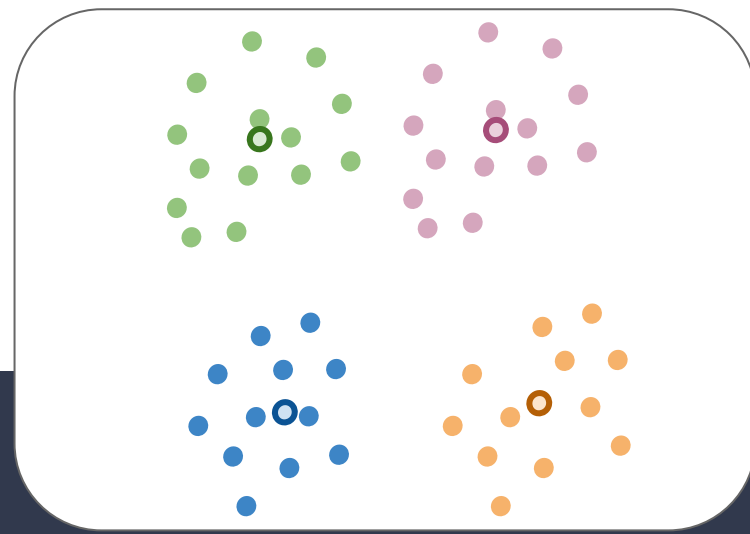
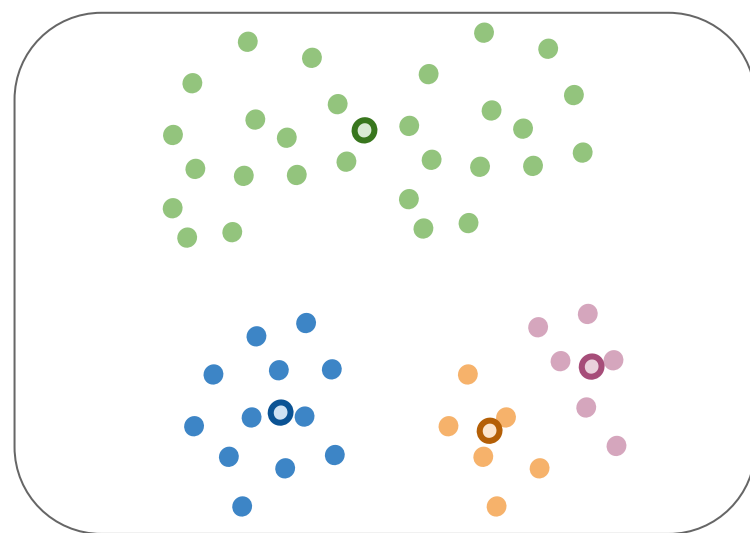
Clustering por prototipos (k-means) > Inicialización

Dos posibles soluciones obtenidas partiendo de diferentes centroides iniciales



Clustering por prototipos (k-means) > Inicialización

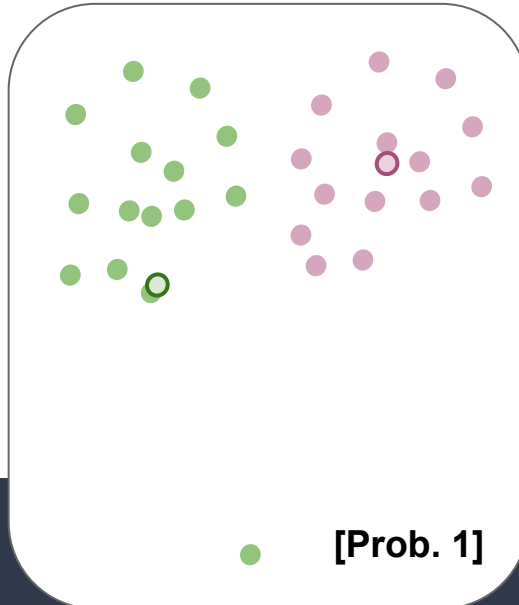
- Elegir al azar y repetir muchas veces, seleccionando la de menor SSE.
- Centroides iniciales cercanos tienden a que k-means converja a un mínimo local pobre -> Elegir un punto al azar, y luego ir eligiendo puntos alejados.
 - *k-means++* (default de *scikit-learn*):
Esos nuevos puntos se eligen con probabilidad $\sim$ 'distancia a los centros anteriores' 2
- Alternativa: [Bisecting k-means](#). Algoritmo híbrido que combina K-Means tradicional con la agrupación jerárquica divisiva. En lugar de generar todos los grupos a la vez, construye una jerarquía de clústeres dividiendo el conjunto de datos de forma progresiva: Se parte de $k=2$, y se repiten rondas de $k=2$ en los clusters de peor SSE (se los divide en 2), hasta alcanzar el k deseado. Más rápido y robusto frente a la inicialización.



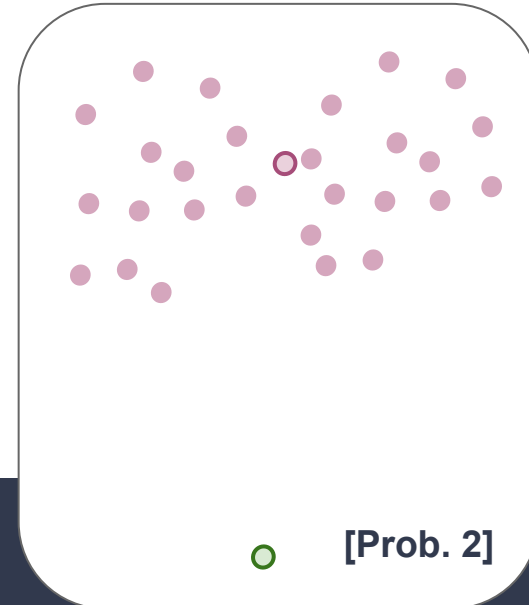
¿Cómo se comporta frente a outliers?



Sesgo de la media frente a outliers:
prototipos sesgados



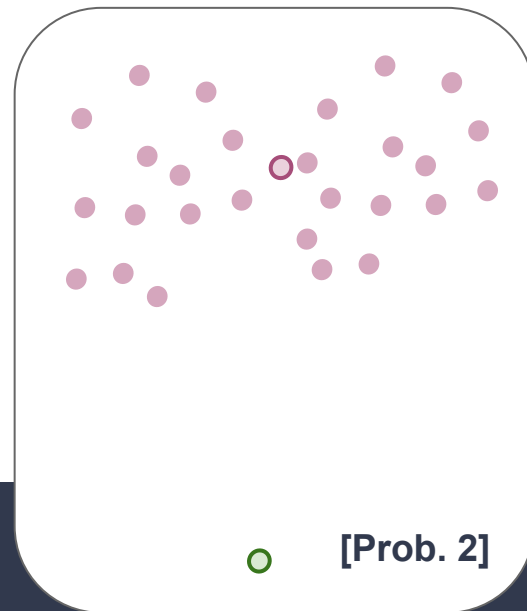
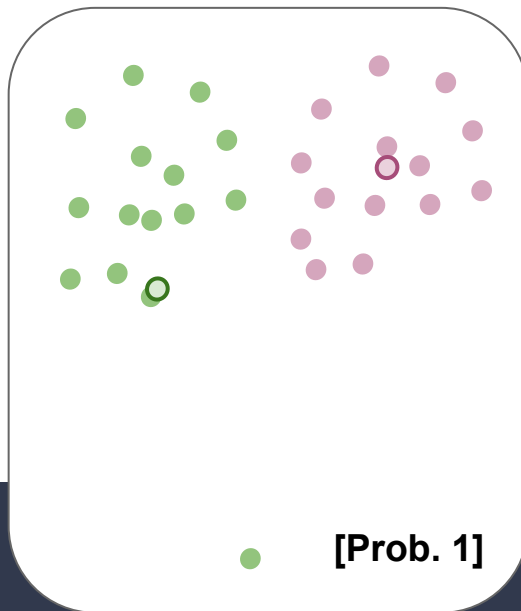
Mala inicialización puede llevar a
resultados aberrantes



Clustering por prototipos (k-means) > Outliers

- Eliminar outliers en el preprocesamiento [Problema 1 y 2].
- Eliminar outliers en el postprocesamiento [Problema 1]
- *k-means++* u otro método de inicialización “inteligente” [Problema 2]
- *k-medoids* (PAM) [Problema 1]

*Elementos que más aportan al error >
SSE o técnicas de validación*



Clustering por prototipos (PAM, Partition around Medoids)

0. Seleccionar K.

1. Seleccionar K ítems como medoides iniciales.
2. Calcular la matriz de distancias.
3. Asignar cada ítem a uno de los K clusters.
4. Computar la sumatoria de las distancias entre los miembros y el medoide; calcular el total entre clusters.

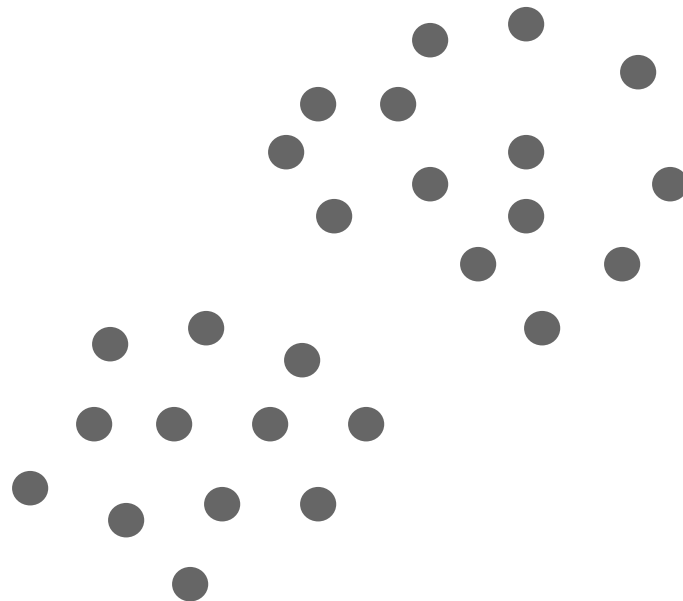
Repetir:

5. Intercambiar el medoide dentro del cluster.
6. Recomputar (3) de cada cluster.
7. Si (4) disminuye, reemplazar el medoide.

Hasta que: los medoides no cambien

Etapa de construcción
(Build phase)

Etapa de intercambio
(Swap phase)



Clustering por prototipos (PAM, Partition around Medoids)

0. Seleccionar K.

- 1. Seleccionar K ítems como medoides iniciales.
- 2. Calcular la matriz de distancias.
- 3. Asignar cada ítem a uno de los K clusters.
- 4. Computar la sumatoria de las distancias entre los miembros y el medoide; calcular el total entre clusters.

Repetir:

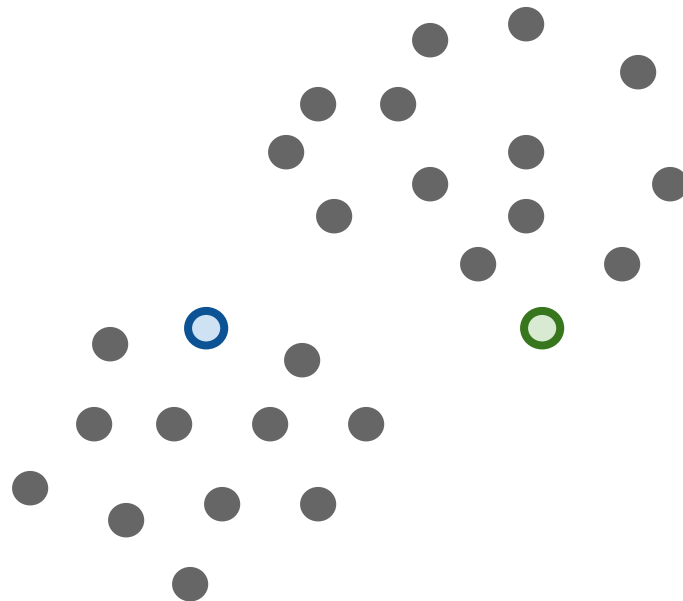
- 5. Intercambiar el medoide dentro del cluster.
- 6. Recomputar (3) de cada cluster.
- 7. Si (4) disminuye, reemplazar el medoide.

Hasta que: los medoides no cambien

Etapas de construcción
(Build phase)

Etapas de intercambio
(Swap phase)

Obs: El medoide **siempre** va a ser un punto de los datos



Clustering por prototipos (PAM, Partition around Medoids)

0. Seleccionar K.

1. Seleccionar K ítems como medoides iniciales.

▶ 2. Calcular la matriz de distancias.

3. Asignar cada ítem a uno de los K clusters.

4. Computar la sumatoria de las distancias entre los miembros y el medoide; calcular el total entre clusters.

Repetir:

5. Intercambiar el medoide dentro del cluster.

6. Recomputar (3) de cada cluster.

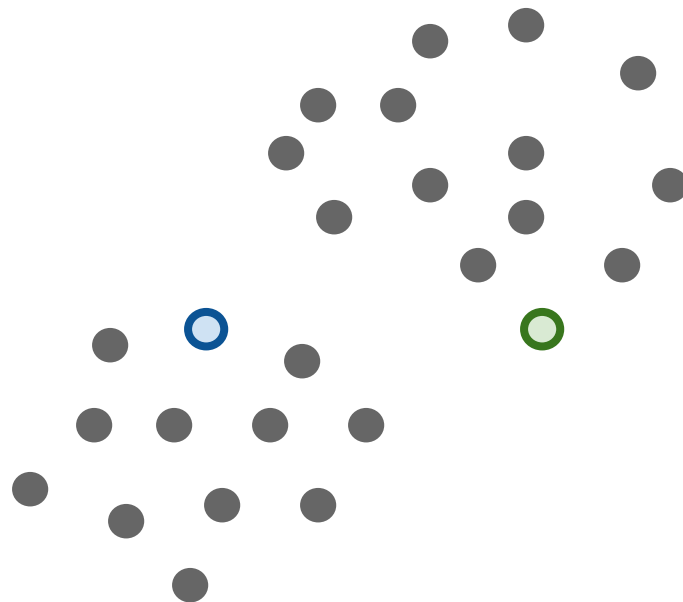
7. Si (4) disminuye, reemplazar el medoide.

Hasta que: los medoides no cambien

Etapas de construcción
(Build phase)

Etapas de intercambio
(Swap phase)

Obs: Se pueden usar diferentes medidas de distancias (no solo la euclídeana)



Clustering por prototipos (PAM, Partition around Medoids)

0. Seleccionar K.
1. Seleccionar K ítems como medoides iniciales.
2. Calcular la matriz de distancias.
- ▶ 3. Asignar cada ítem a uno de los K clusters.
4. Computar la sumatoria de las distancias entre los miembros y el medoide; calcular el total entre clusters.

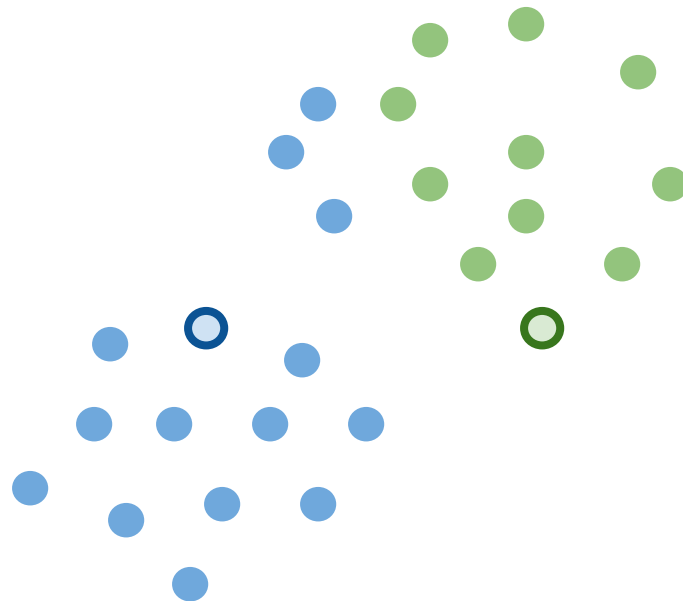
Repetir:

5. Intercambiar el medoide dentro del cluster.
6. Recomputar (3) de cada cluster.
7. Si (4) disminuye, reemplazar el medoide.

Hasta que: los medoides no cambien

Etapas de construcción
(Build phase)

Etapas de intercambio
(Swap phase)



Clustering por prototipos (PAM, Partition around Medoids)

0. Seleccionar K.
1. Seleccionar K ítems como medoides iniciales.
2. Calcular la matriz de distancias.
3. Asignar cada ítem a uno de los K clusters.
- ▶ 4. Computar la sumatoria de las distancias entre los miembros y el medoide; calcular el total entre clusters.

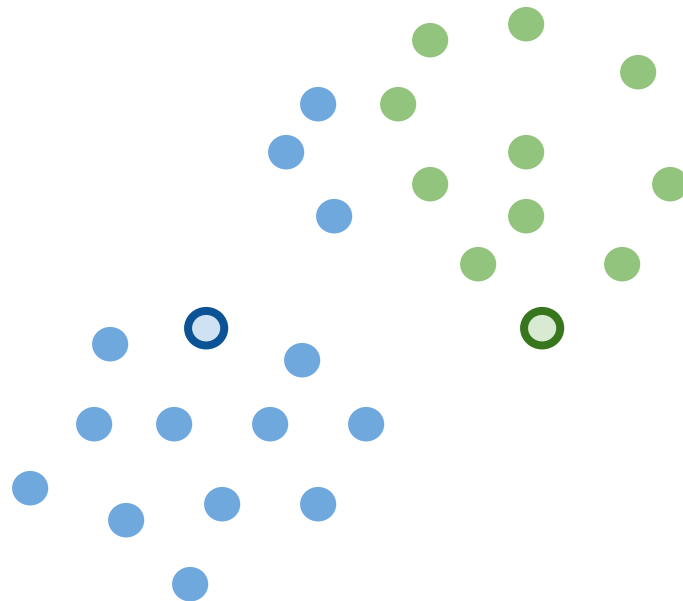
Repetir:

5. Intercambiar el medoide dentro del cluster.
6. Recomputar (3) de cada cluster.
7. Si (4) disminuye, reemplazar el medoide.

Hasta que: los medoides no cambien

Etapas de construcción
(Build phase)

Etapas de intercambio
(Swap phase)



Clustering por prototipos (PAM, Partition around Medoids)

0. Seleccionar K.
1. Seleccionar K ítems como medoides iniciales.
2. Calcular la matriz de distancias.
3. Asignar cada ítem a uno de los K clusters.
4. Computar la sumatoria de las distancias entre los miembros y el medoide; calcular el total entre clusters.

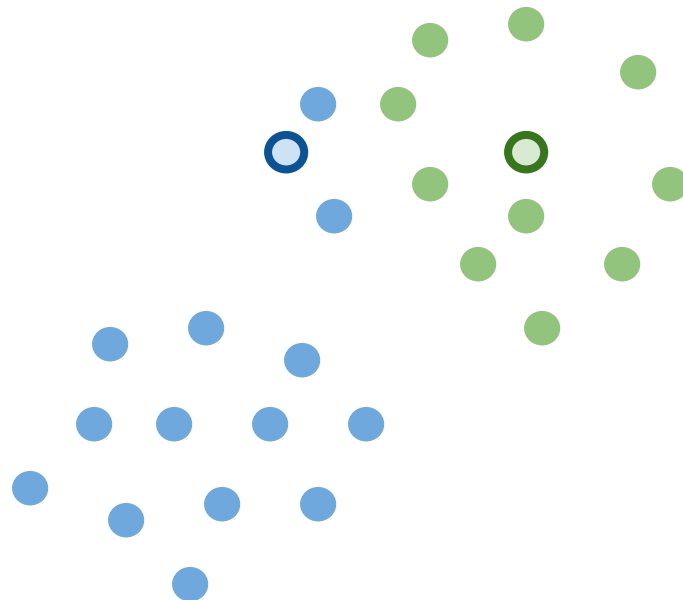
Repetir:

5. Intercambiar el medoide dentro del cluster.
6. Recomputar (3) de cada cluster.
7. Si (4) disminuye, reemplazar el medoide.

Hasta que: los medoides no cambien

Etapas de construcción
(Build phase)

Etapas de intercambio
(Swap phase)



Clustering por prototipos (PAM, Partition around Medoids)

0. Seleccionar K.
1. Seleccionar K ítems como medoides iniciales.
2. Calcular la matriz de distancias.
3. Asignar cada ítem a uno de los K clusters.
4. Computar la sumatoria de las distancias entre los miembros y el medoide; calcular el total entre clusters.

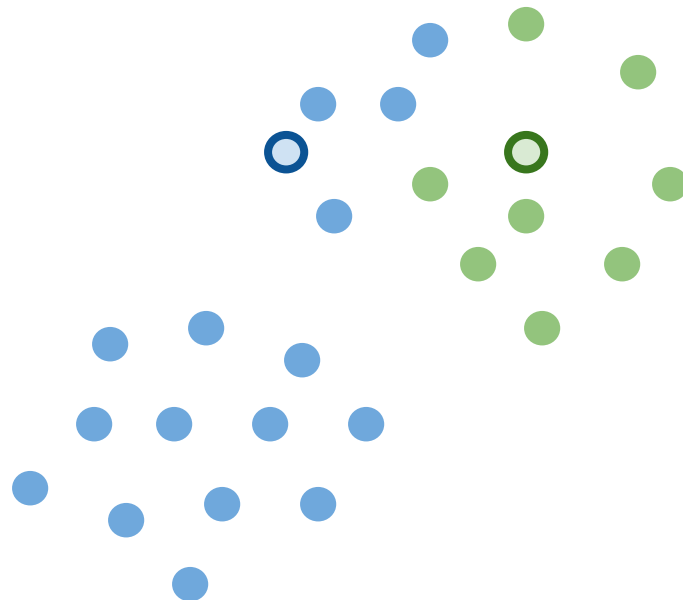
Repetir:

5. Intercambiar el medoide dentro del cluster.
6. Recomputar (3) de cada cluster.
7. Si (4) disminuye, reemplazar el medoide.

Hasta que: los medoides no cambien

Etapas de construcción
(Build phase)

Etapas de intercambio
(Swap phase)



Clustering por prototipos (PAM, Partition around Medoids)

0. Seleccionar K.
1. Seleccionar K ítems como medoides iniciales.
2. Calcular la matriz de distancias.
3. Asignar cada ítem a uno de los K clusters.
4. Computar la sumatoria de las distancias entre los miembros y el medoide; calcular el total entre clusters.

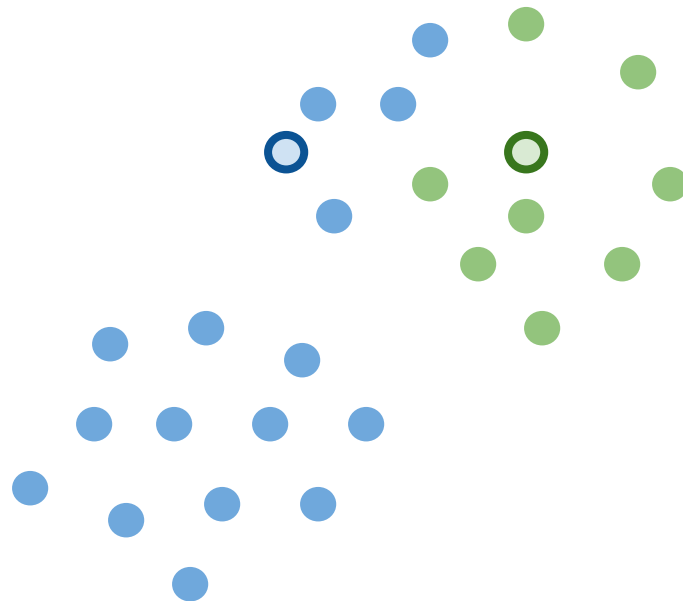
Repetir:

5. Intercambiar el medoide dentro del cluster.
6. Recomputar (3) de cada cluster.
7. Si (4) disminuye, reemplazar el medoide.

Hasta que: los medoides no cambien

Etapas de construcción
(Build phase)

Etapas de intercambio
(Swap phase)



Clustering por prototipos (PAM, Partition around Medoids)

0. Seleccionar K.

1. Seleccionar K ítems como medoides iniciales.
2. Calcular la matriz de distancias.
3. Asignar cada ítem a uno de los K clusters.
4. Computar la sumatoria de las distancias entre los miembros y el medoide; calcular el total entre clusters.

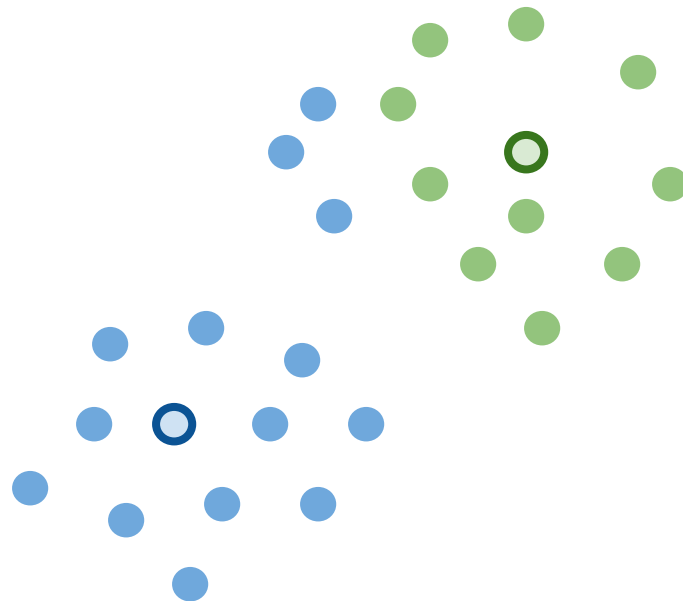
Repetir:

5. Intercambiar el medoide dentro del cluster.
6. Recomputar (3) de cada cluster.
7. Si (4) disminuye, reemplazar el medoide.

Hasta que: los medoides no cambien

Etapas de construcción
(Build phase)

Etapas de intercambio
(Swap phase)



Clustering por prototipos (PAM, Partition around Medoids)

0. Seleccionar K.
1. Seleccionar K ítems como medoides iniciales.
2. Calcular la matriz de distancias.
3. Asignar cada ítem a uno de los K clusters.
4. Computar la sumatoria de las distancias entre los miembros y el medoide; calcular el total entre clusters.

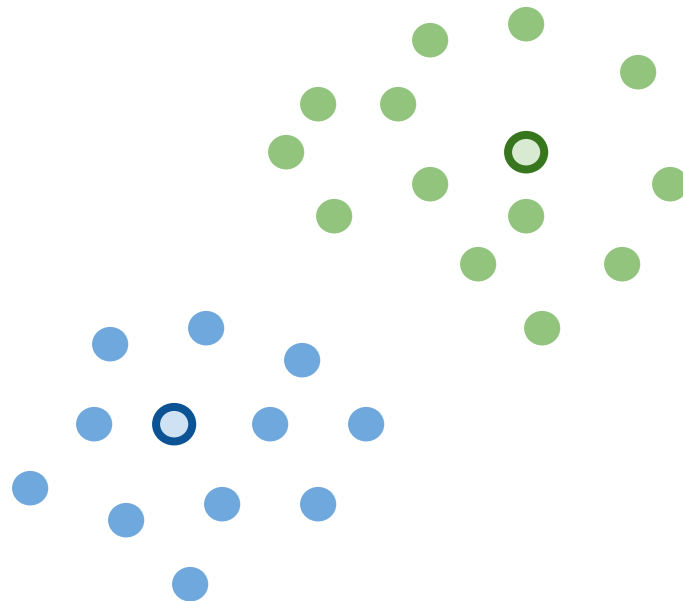
Repetir:

5. Intercambiar el medoide dentro del cluster.
6. Recomputar (3) de cada cluster.
7. Si (4) disminuye, reemplazar el medoide.

Hasta que: los medoides no cambien

Etapa de construcción
(Build phase)

Etapa de intercambio
(Swap phase)



Clustering por prototipos (PAM, Partition around Medoids)

0. Seleccionar K.
1. Seleccionar K ítems como medoides iniciales.
2. Calcular la matriz de distancias.
3. Asignar cada ítem a uno de los K clusters.
4. Computar la sumatoria de las distancias entre los miembros y el medoide; calcular el total entre clusters.

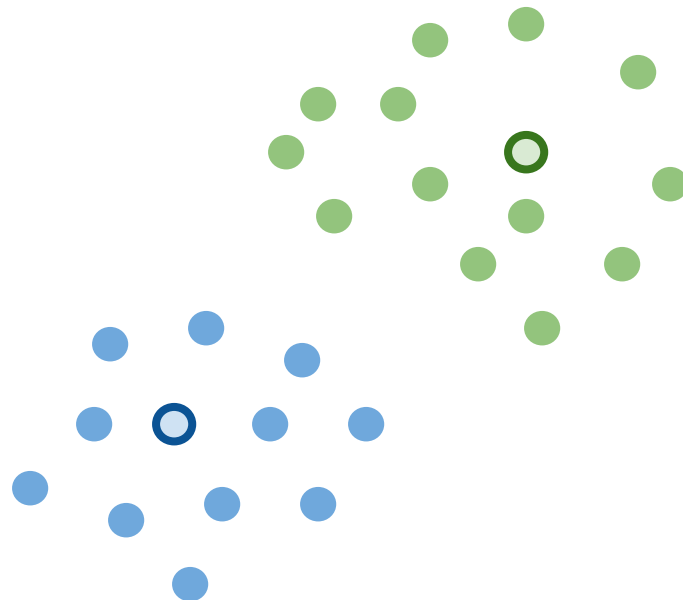
Repetir:

5. Intercambiar el medoide dentro del cluster.
6. Recomputar (3) de cada cluster.
7. Si (4) disminuye, reemplazar el medoide.

Hasta que: los medoides no cambien

Etapas de construcción
(Build phase)

Etapas de intercambio
(Swap phase)



Clustering por prototipos (PAM, Partition around Medoids)

0. Seleccionar K.

1. Seleccionar K ítems como medoides iniciales.
2. Calcular la matriz de distancias.
3. Asignar cada ítem a uno de los K clusters.
4. Computar la sumatoria de las distancias entre los miembros y el medoide; calcular el total entre clusters.

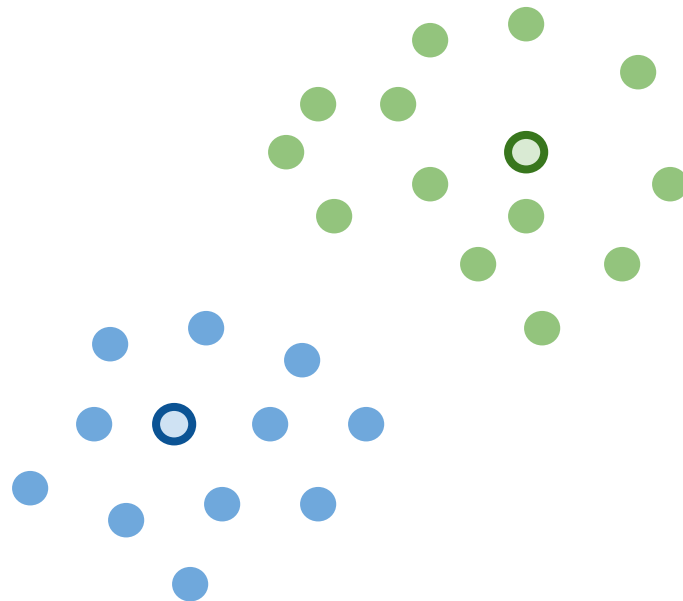
Repetir:

5. Intercambiar el medoide dentro del cluster.
6. Recomputar (3) de cada cluster.
7. Si (4) disminuye, reemplazar el medoide.

Hasta que: los medoides no cambien

Etapa de construcción
(Build phase)

Etapa de intercambio
(Swap phase)



Clustering por prototipos (PAM, Partition around Medoids)

0. Seleccionar K.
1. Seleccionar K ítems como medoides iniciales.
2. Calcular la matriz de distancias.
3. Asignar cada ítem a uno de los K clusters.
4. Computar la sumatoria de las distancias entre los miembros y el medoide; calcular el total entre clusters.

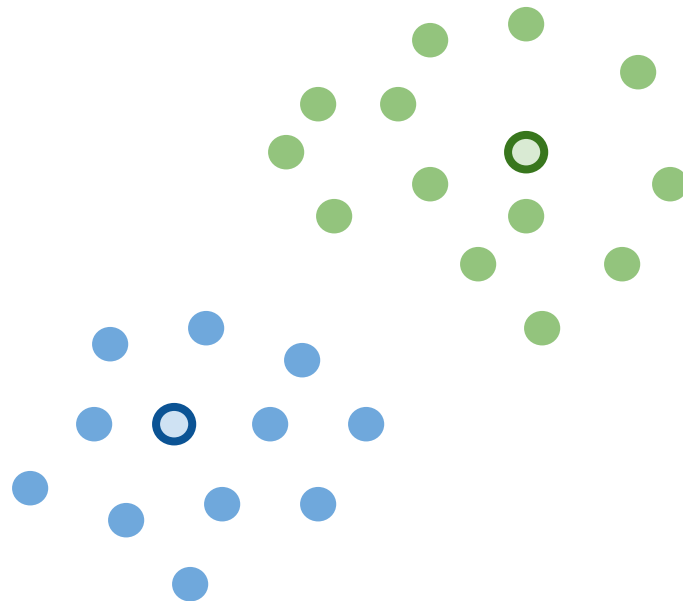
Repetir:

5. Intercambiar el medoide dentro del cluster.
6. Recomputar (3) de cada cluster.
7. Si (4) disminuye, reemplazar el medoide.

Hasta que: los medoides no cambien

Etapa de construcción
(Build phase)

Etapa de intercambio
(Swap phase)



Clustering por prototipos (PAM, Partition around Medoids)

0. Seleccionar K.
1. Seleccionar K ítems como medoides iniciales.
2. Calcular la matriz de distancias.
3. Asignar cada ítem a uno de los K clusters.
4. Computar la sumatoria de las distancias entre los miembros y el medoide; calcular el total entre clusters.

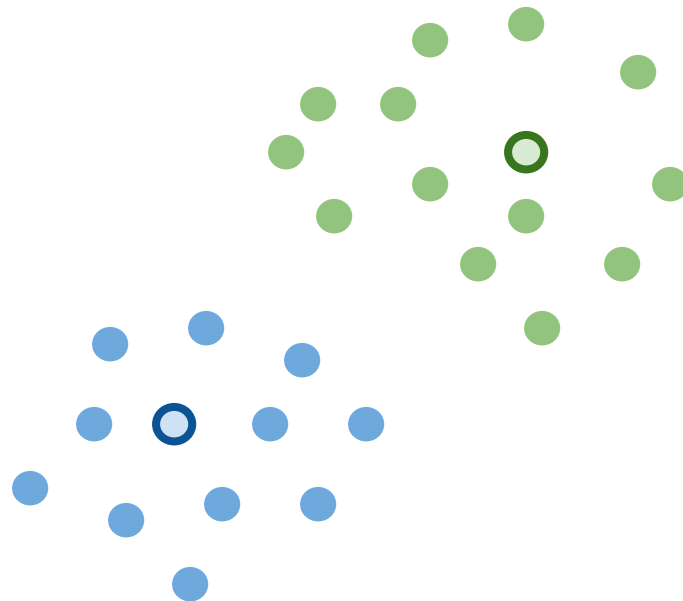
Repetir:

5. Intercambiar el medoide dentro del cluster.
6. Recomputar (3) de cada cluster.
7. Si (4) disminuye, reemplazar el medoide.

Hasta que: los medoides no cambien

Etapa de construcción
(Build phase)

Etapa de intercambio
(Swap phase)



Clustering por prototipos (PAM, Partition around Medoids)

0. Seleccionar K.
1. Seleccionar K ítems como medoides iniciales.
2. Calcular la matriz de distancias.
3. Asignar cada ítem a uno de los K clusters.
4. Computar la sumatoria de las distancias entre los miembros y el medoide; calcular el total entre clusters.

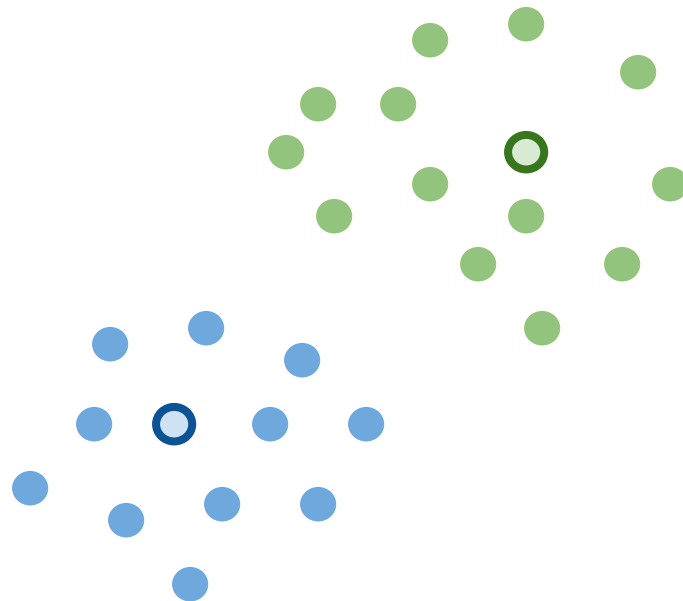
Repetir:

5. Intercambiar el medoide dentro del cluster.
6. Recomputar (3) de cada cluster.
7. Si (4) disminuye, reemplazar el medoide.

▶ **Hasta que:** los medoides no cambien

Etapa de construcción
(Build phase)

Etapa de intercambio
(Swap phase)



Clustering por prototipos (PAM, Partition around Medoids)

0. Seleccionar K.

1. Seleccionar K ítems como medoides iniciales.
2. Calcular la matriz de distancias.
3. Asignar cada ítem a uno de los K clusters.
4. Computar la sumatoria de las distancias entre los miembros y el medoide; calcular el total entre clusters.

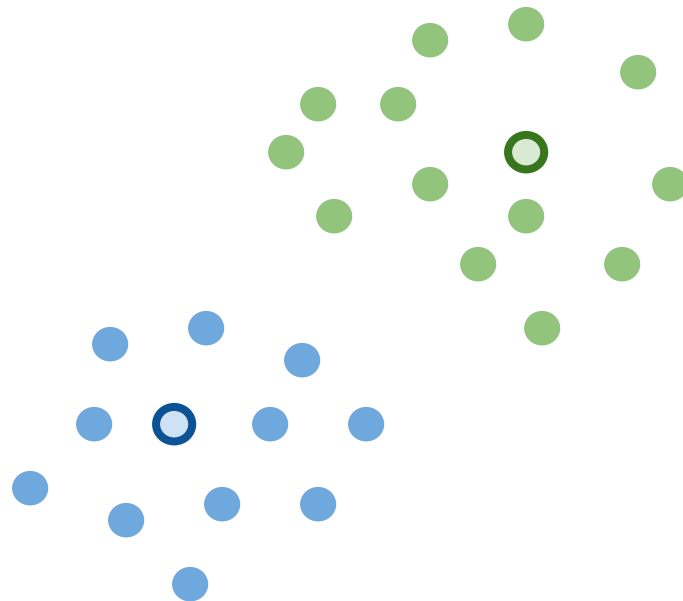
Repetir:

5. Intercambiar el medoide dentro del cluster.
6. Recomputar (3) de cada cluster.
7. Si (4) disminuye, reemplazar el medoide.

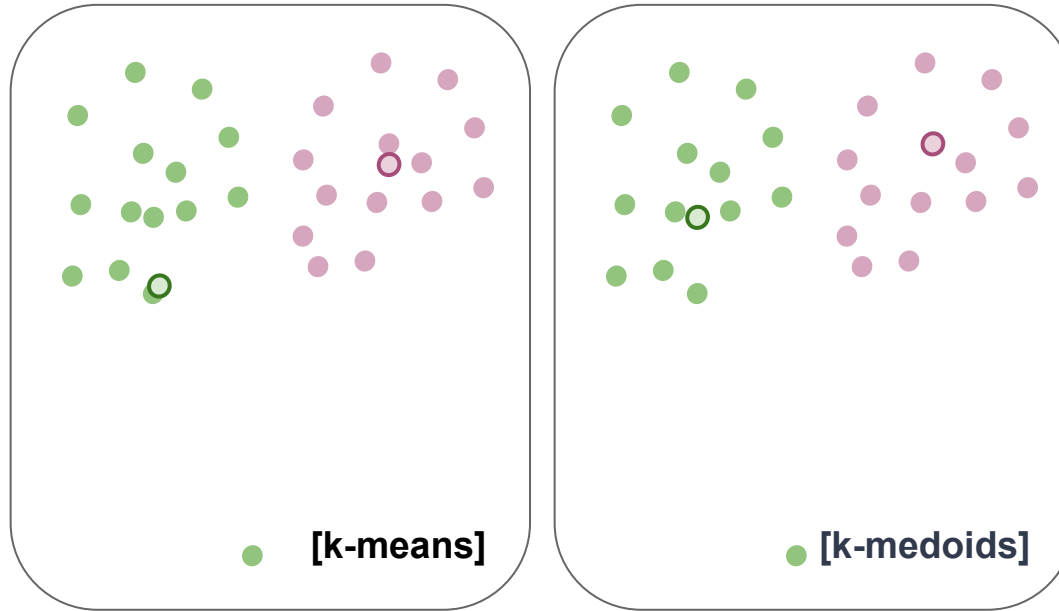
Hasta que: los medoides no cambien

Etapas de construcción
(Build phase)

Etapas de intercambio
(Swap phase)



Clustering por prototipos (k-medoids) > Outliers



Clustering por prototipos k-means vs k-medoids

k-means

La distancia es la *Euclídea*, y la función objetivo:

$$SSE = \sum_{i=1}^K \sum_{x \in C_i} d^2(x - x_{c_i}) = \sum_{i=1}^K \sum_{x \in C_i} \|x - x_{c_i}\|^2$$

Complejidad

n = número de puntos, d = número de atributos,
 k = número de clusters, m = número iteraciones

Si se fijan los parámetros n , k , d , $m \Rightarrow$

$$O(n * k * d * m) \sim O(n * k * \dots)$$

k-medoids

Menos sensible a outliers

Más flexible para otras distancias

Complejidad

n = número de puntos, d = número de atributos,
 k = número de clusters, m = número iteraciones

Si se fijan los parámetros n , k , d , $m \Rightarrow$

$$O(k * (n - k)^2 * \dots)$$

No escala bien para n grandes.

Se puede pre-computar la matriz de distancias, pero dependiendo del n puede ser muy demandante en memoria.



Clustering por prototipos (PAM, Partition around Medoids)

- Implementación en scikit-learn:

https://scikit-learn-extra.readthedocs.io/en/latest/generated/sklearn_extra.cluster.KMedoids.html#sklearn_extra.cluster.KMedoids

- Implementación a mano:

https://github.com/salspaugh/machine_learning/blob/master/clustering/kmedoids.py

Fallas típicas en *k-means*

(y en alguna medida de los métodos por prototipo)

k-means funciona muy bien en la presencia de clusters “esféricos” de similar dispersión. Pero...

k-means solo usa distancia entre el prototipo y las muestras: No hay representación del tamaño, dispersión y/o forma del cluster.

Resultado: **potenciales errores**

Fallas típicas en *k-means*

Clusters de distinto tamaño
(penaliza el error del cluster grande y trata de compensar corriendo el prototipo)

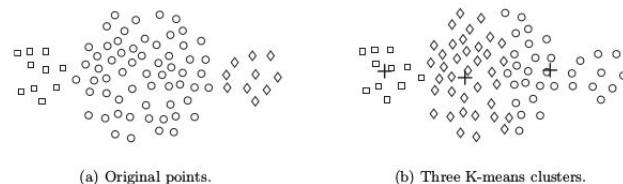


Figure 8.9. K-means with clusters of different size.

Clusters con distinta densidad
(penaliza el error del cluster menos denso y lo divide)

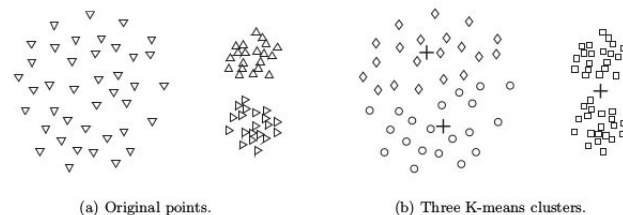


Figure 8.10. K-means with clusters of different density.

Clusters no globulares
(no “interpreta” bien clusters no globulares y los divide)

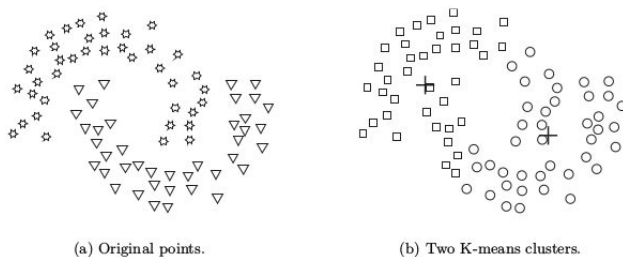
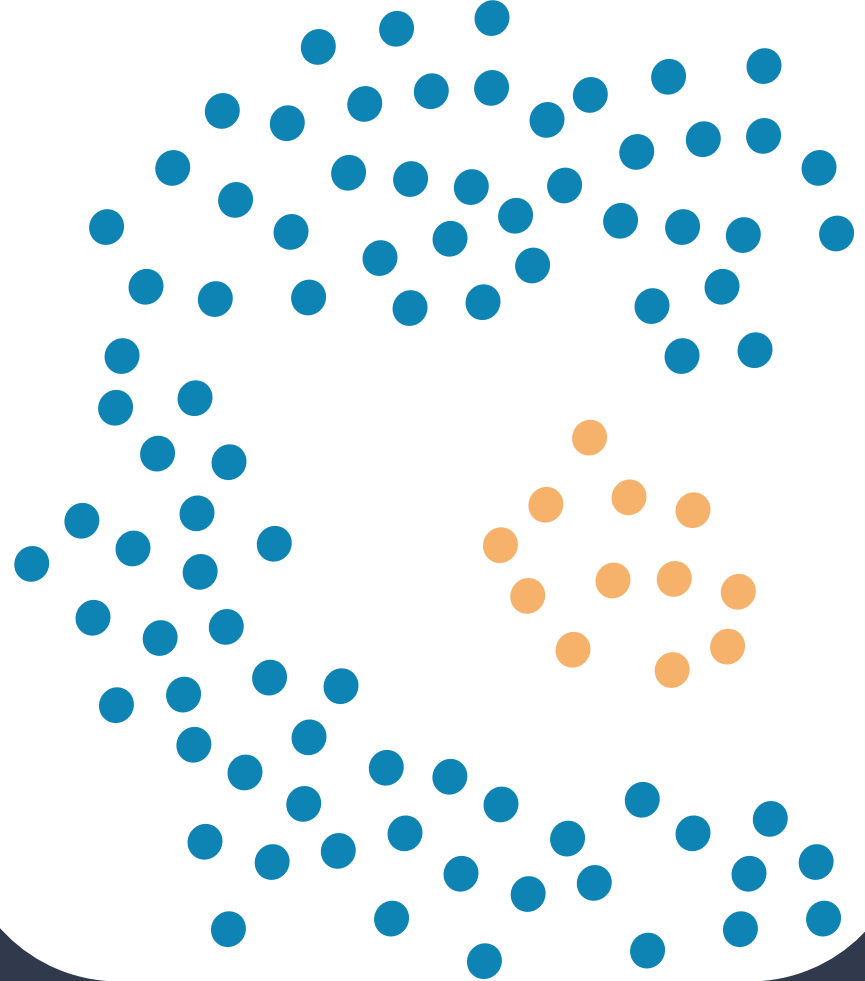
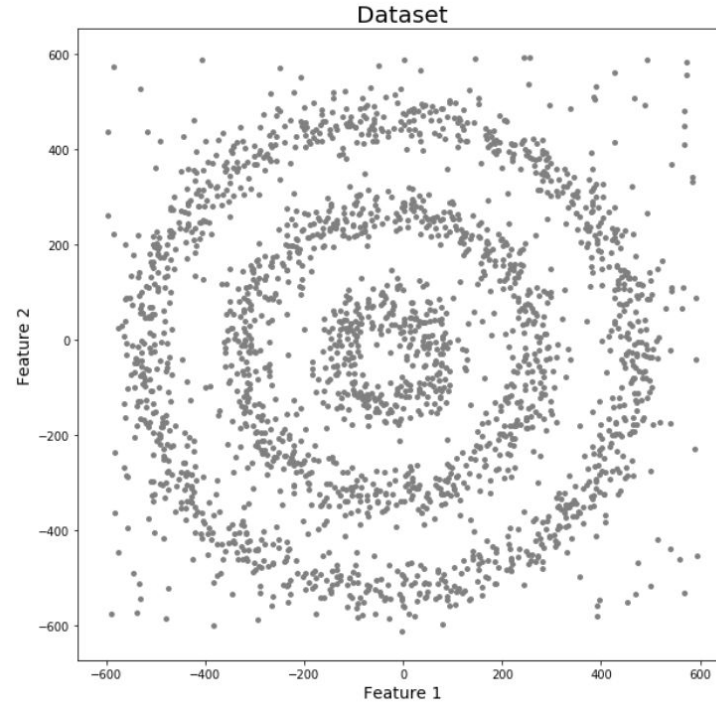


Figure 8.11. K-means with non-globular clusters.

Para evaluar la efectividad que podría tener un algoritmo de clustering, siempre **PRIMERO VISUALIZACIÓN**



¿Y qué hago con esto? ¿Me sigo basando en centroides que sirvan de prototipo?



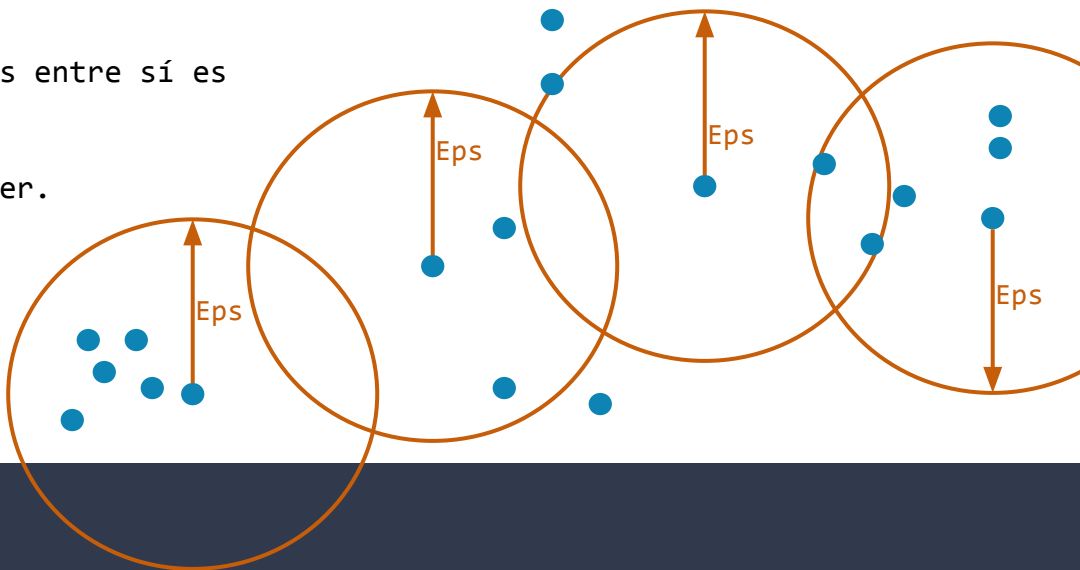
Clustering por densidad (DBSCAN)

0. Elegir valores para los parámetros *Eps* y *MinPts*.
1. Identificar todos los elementos como Semilla, Borde o Ruido.
2. Eliminar Ruido.
3. Unir todos los elementos Semilla que se encuentren a menos de distancia *Eps*.
4. Cada grupo de elementos Semilla conectados entre sí es un cluster.
5. Asignar los elementos Borde a algún cluster.

Semilla = Core

Borde = Border

Ruido = Noise

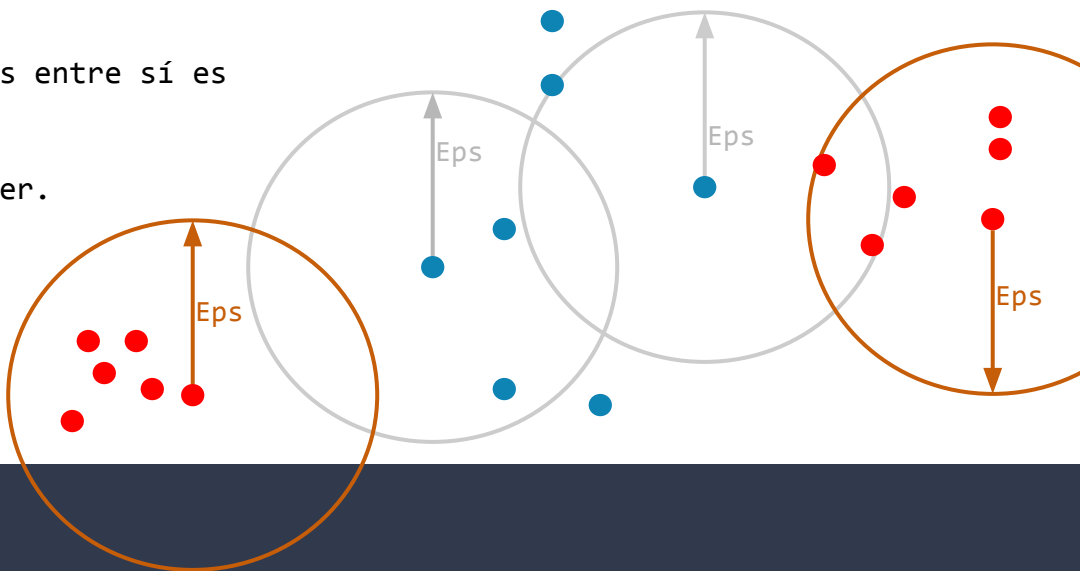


Clustering por densidad (DBSCAN)

Es **semilla** si tiene al menos **MinPts** dentro de un radio **Eps**.

0. Elegir valores para los parámetros *Eps* y *MinPts*.
1. Identificar todos los elementos como Semilla, Borde o Ruido.
2. Eliminar Ruido.
3. Unir todos los elementos Semilla que se encuentren a menos de distancia *Eps*.
4. Cada grupo de elementos Semilla conectados entre sí es un cluster.
5. Asignar los elementos Borde a algún cluster.

Ej. *Eps*, *MinPts* = 4



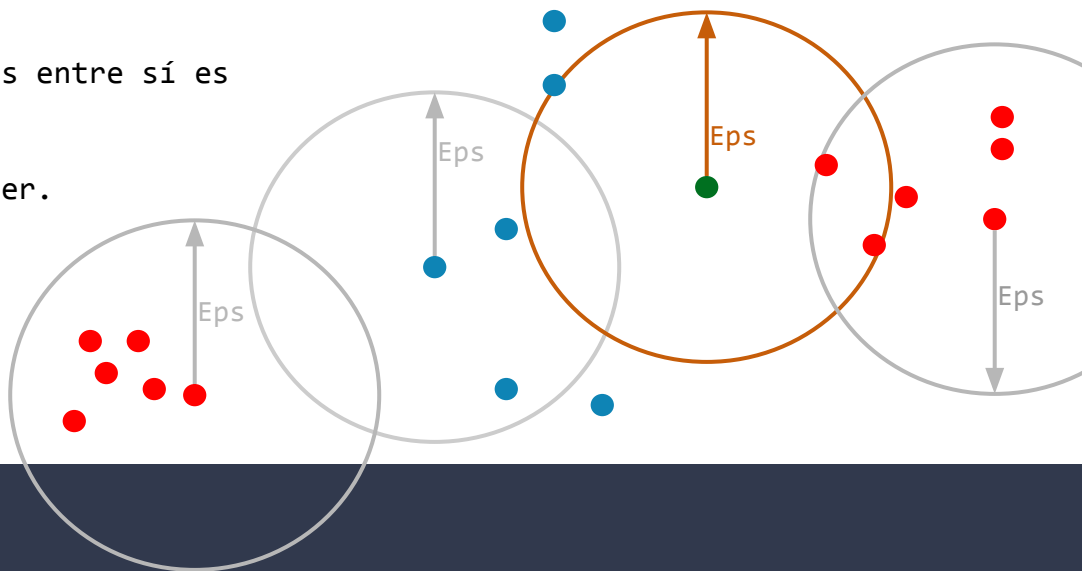
Clustering por densidad (DBSCAN)

0. Elegir valores para los parámetros *Eps* y *MinPts*.
1. Identificar todos los elementos como Semilla, Borde o Ruido.
2. Eliminar Ruido.
3. Unir todos los elementos Semilla que se encuentren a menos de distancia *Eps*.
4. Cada grupo de elementos Semilla conectados entre sí es un cluster.
5. Asignar los elementos Borde a algún cluster.

Es **semilla** si tiene al menos **MinPts** dentro de un radio **Eps**.

Es **borde** si tiene menos de **MinPts** dentro de un radio **Eps**, y está conectado con puntos semilla.

Ej. *Eps*, *MinPts* = 4



Clustering por densidad (DBSCAN)

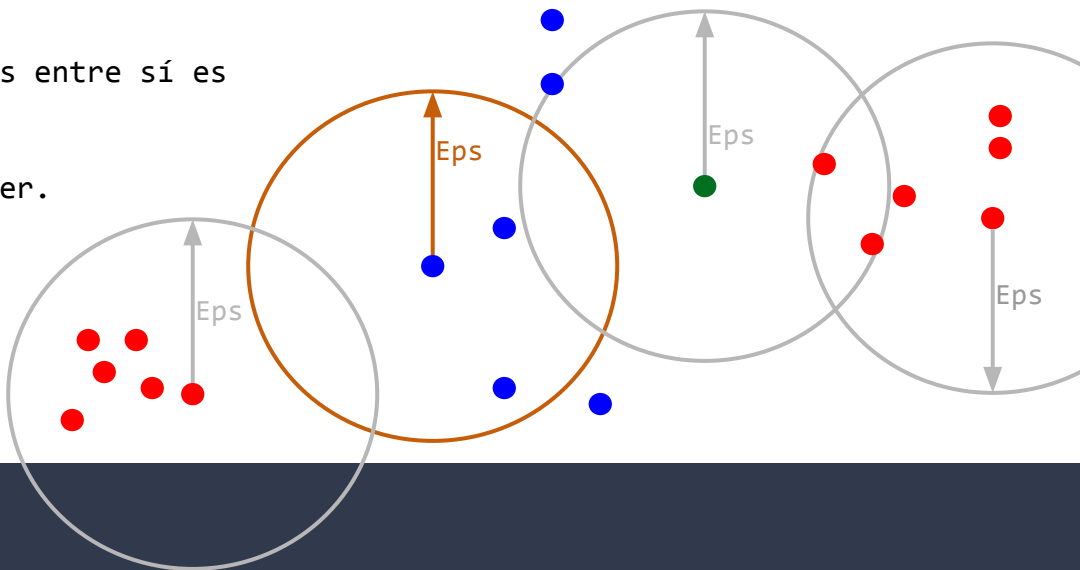
0. Elegir valores para los parámetros *Eps* y *MinPts*.
1. Identificar todos los elementos como Semilla, Borde o Ruido.
2. Eliminar Ruido.
3. Unir todos los elementos Semilla que se encuentren a menos de distancia *Eps*.
4. Cada grupo de elementos Semilla conectados entre sí es un cluster.
5. Asignar los elementos Borde a algún cluster.

Es **semilla** si tiene al menos **MinPts** dentro de un radio **Eps**.

Es **borde** si tiene menos de **MinPts** dentro de un radio **Eps**, y está conectado con puntos semilla.

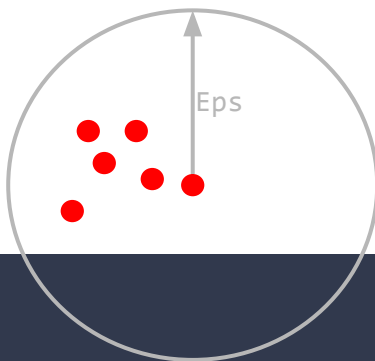
Es **ruido** si tiene menos de **MinPts** dentro de un radio **Eps**, y no está conectado con puntos semilla.

Ej. *Eps*, *MinPts* = 4



Clustering por densidad (DBSCAN)

0. Elegir valores para los parámetros *Eps* y *MinPts*.
1. Identificar todos los elementos como Semilla, Borde o Ruido.
2. Eliminar Ruido.
3. Unir todos los elementos Semilla que se encuentren a menos de distancia *Eps*.
4. Cada grupo de elementos Semilla conectados entre sí es un cluster.
5. Asignar los elementos Borde a algún cluster.

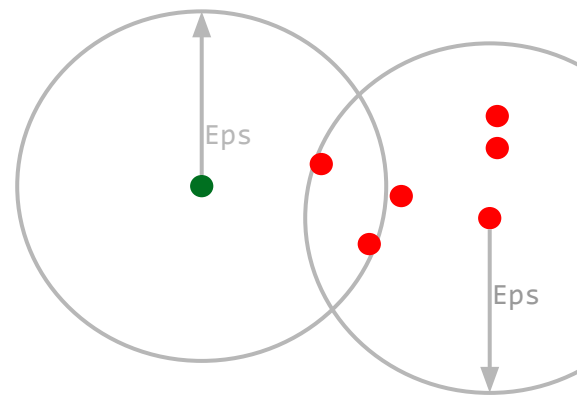


Es **semilla** si tiene al menos **MinPts** dentro de un radio **Eps**.

Es **borde** si tiene menos de **MinPts** dentro de un radio **Eps**, y está conectado con puntos semilla.

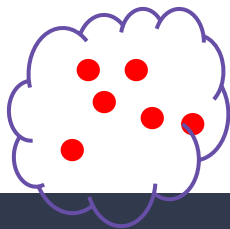
Es **ruido** si tiene menos de **MinPts** dentro de un radio **Eps**, y no está conectado con puntos semilla.

Ej. *Eps*, *MinPts* = 4



Clustering por densidad (DBSCAN)

0. Elegir valores para los parámetros *Eps* y *MinPts*.
1. Identificar todos los elementos como Semilla, Borde o Ruido.
2. Eliminar Ruido.
3. Unir todos los elementos Semilla que se encuentren a menos de distancia *Eps*.
4. Cada grupo de elementos Semilla conectados entre sí es un cluster.
5. Asignar los elementos Borde a algún cluster.

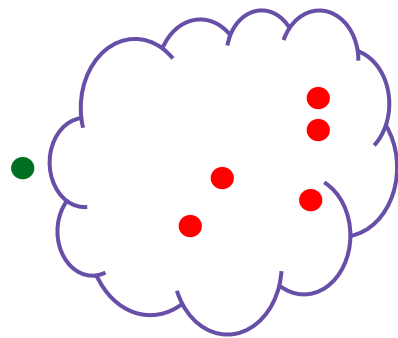


Es **semilla** si tiene al menos **MinPts** dentro de un radio **Eps**.

Es **borde** si tiene menos de **MinPts** dentro de un radio **Eps**, y está conectado con puntos semilla.

Es **ruido** si tiene menos de **MinPts** dentro de un radio **Eps**, y no está conectado con puntos semilla.

Ej. $Eps, MinPts = 4$



Clustering por densidad (DBSCAN)

0. Elegir valores para los parámetros *Eps* y *MinPts*.
1. Identificar todos los elementos como Semilla, Borde o Ruido.
2. Eliminar Ruido.
3. Unir todos los elementos Semilla que se encuentren a menos de distancia *Eps*.
4. Cada grupo de elementos Semilla conectados entre sí es un cluster.
5. Asignar los elementos Borde a algún cluster.

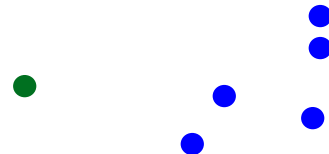


Es **semilla** si tiene al menos **MinPts** dentro de un radio **Eps**.

Es **borde** si tiene menos de **MinPts** dentro de un radio **Eps**, y está conectado con puntos semilla.

Es **ruido** si tiene menos de **MinPts** dentro de un radio **Eps**, y no está conectado con puntos semilla.

Ej. $Eps, MinPts = 4$



Clustering por densidad (DBSCAN)

0. Elegir valores para los parámetros *Eps* y *MinPts*.
1. Identificar todos los elementos como Semilla, Borde o Ruido.
2. Eliminar Ruido.
3. Unir todos los elementos Semilla que se encuentren a menos de distancia *Eps*.
4. Cada grupo de elementos Semilla conectados entre sí es un cluster.
5. Asignar los elementos Borde a algún cluster.

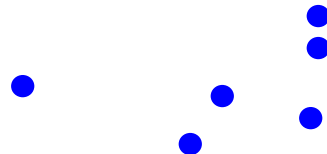


Es **semilla** si tiene al menos **MinPts** dentro de un radio **Eps**.

Es **borde** si tiene menos de **MinPts** dentro de un radio **Eps**, y está conectado con puntos semilla.

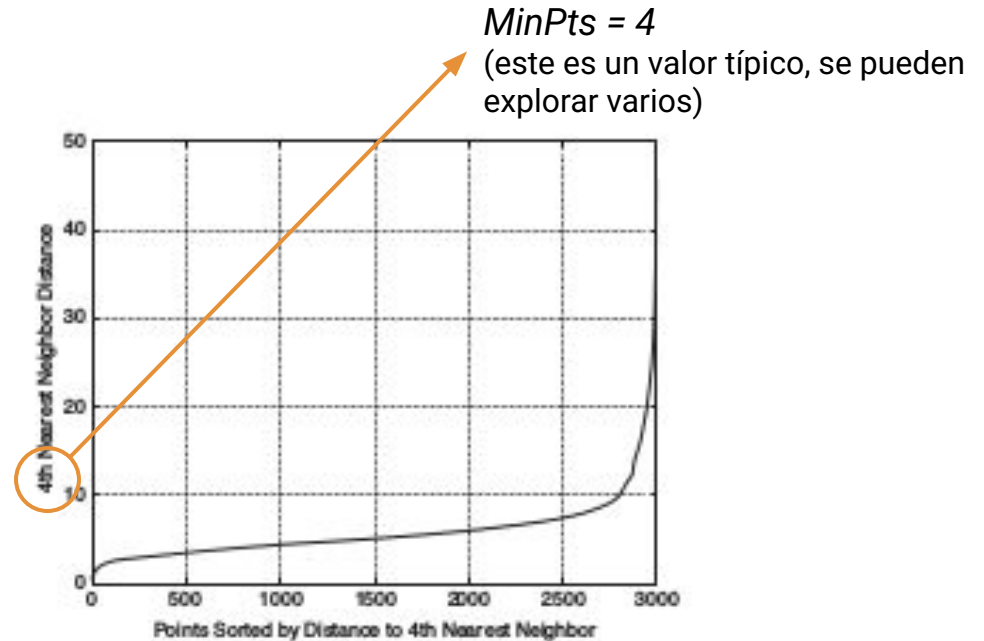
Es **ruido** si tiene menos de **MinPts** dentro de un radio **Eps**, y no está conectado con puntos semilla.

Ej. *Eps*, *MinPts* = 4



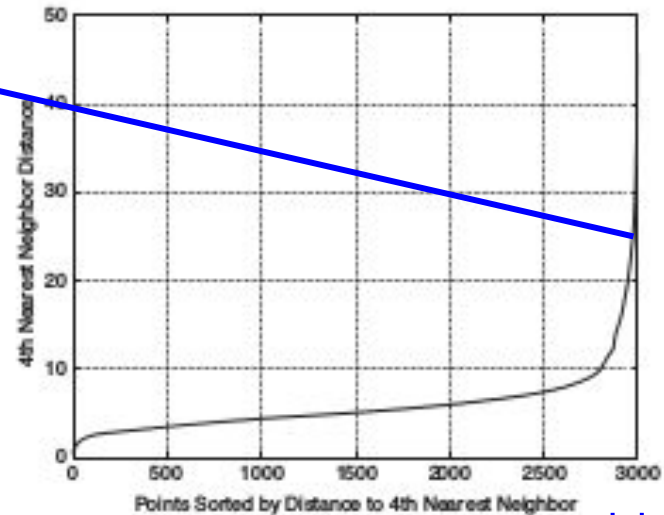
Clustering por densidad (DBSCAN)

0. Elegir valores para los parámetros *Eps* y *MinPts*.



Clustering por densidad (DBSCAN)

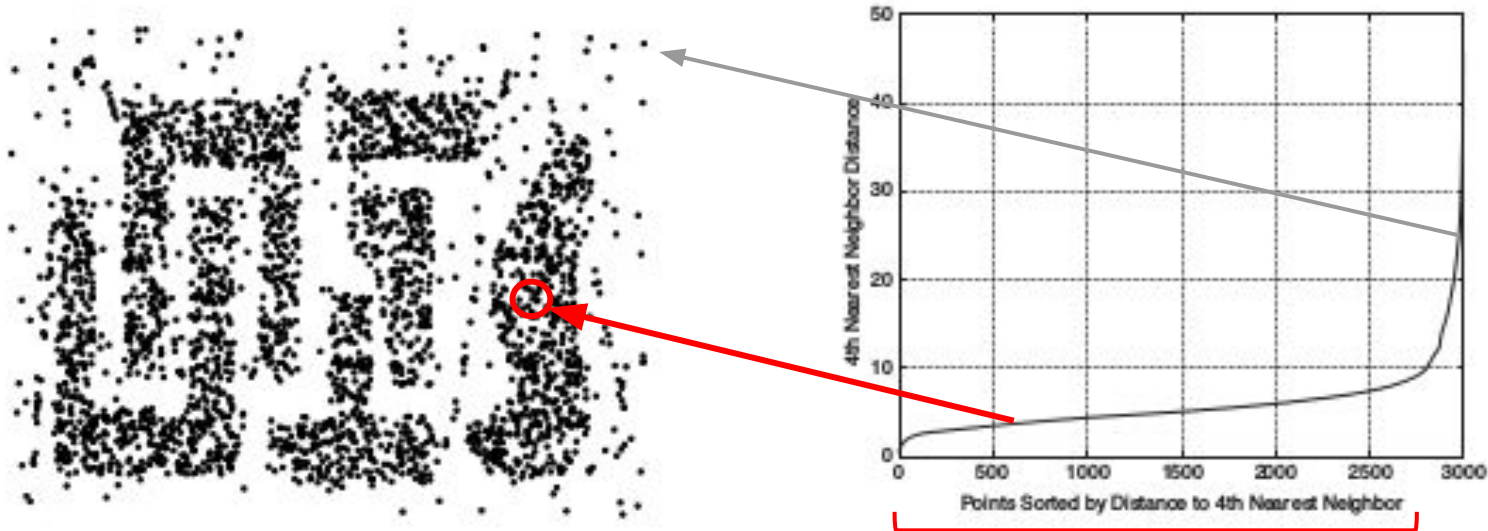
0. Elegir valores para los parámetros *Eps* y *MinPts*.



Ruido = Noise
(Pocos) Puntos que
están lejos del resto

Clustering por densidad (DBSCAN)

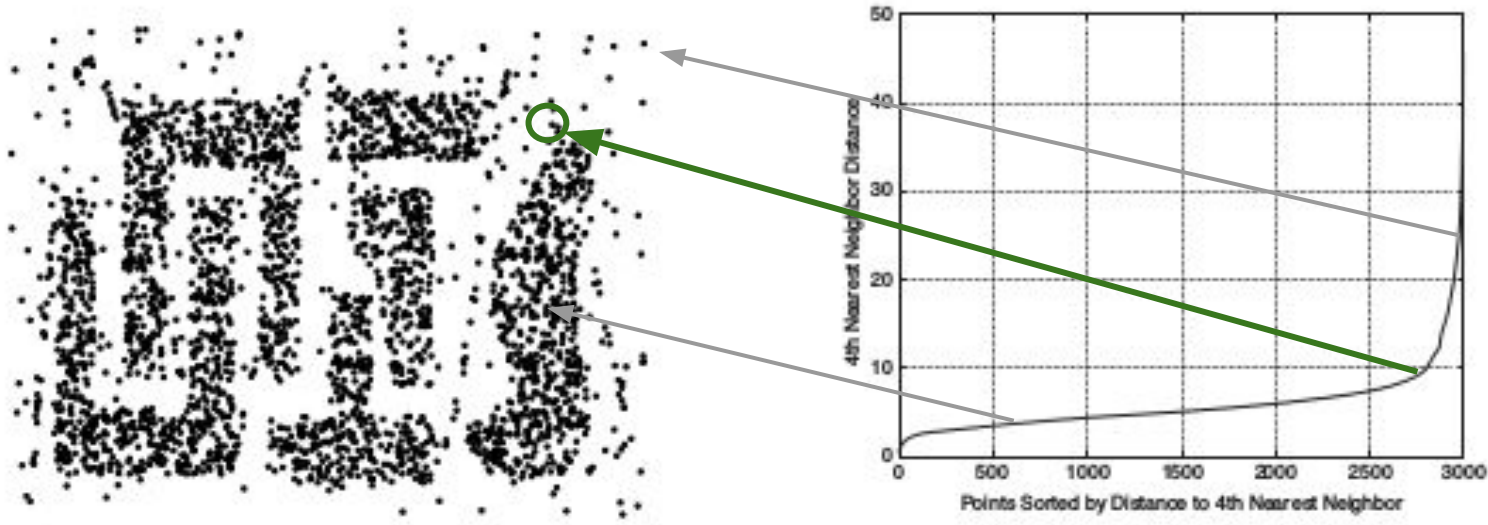
0. Elegir valores para los parámetros *Eps* y *MinPts*.



Semilla = Core
(Muchos) Puntos que
están cerca de otros

Clustering por densidad (DBSCAN)

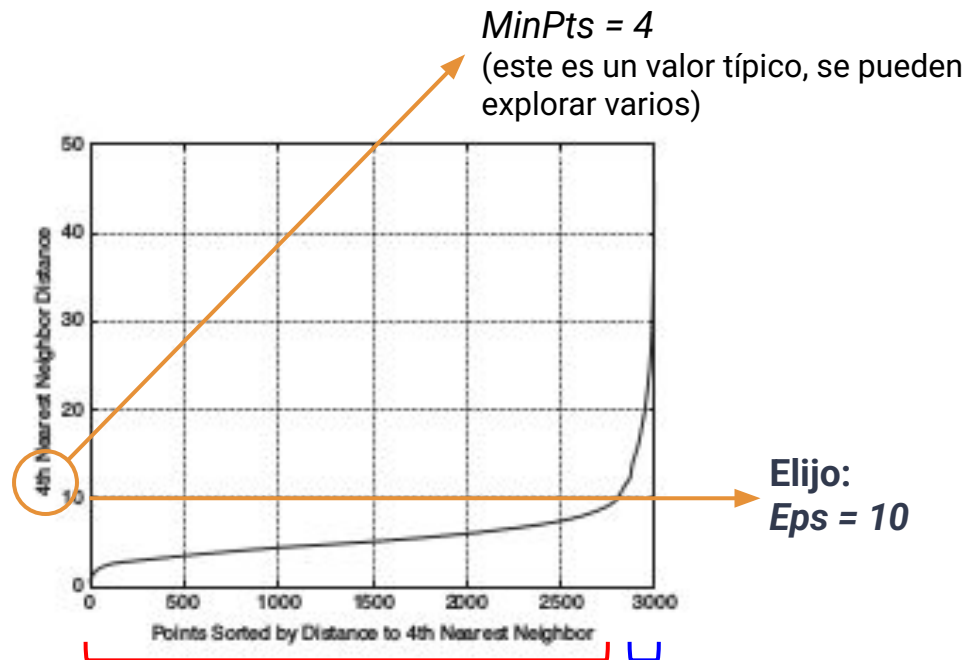
0. Elegir valores para los parámetros *Eps* y *MinPts*.



Borde = Border
(Algunos) Puntos que
están en el límite

Clustering por densidad (DBSCAN)

0. Elegir valores para los parámetros *Eps* y *MinPts*.

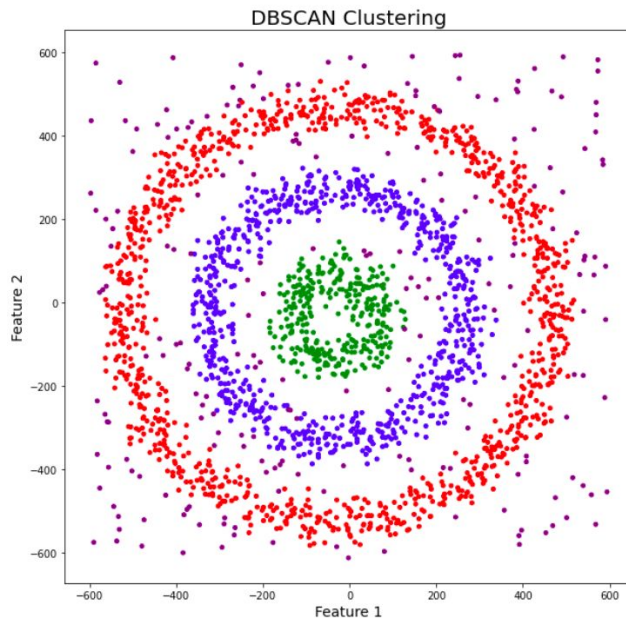


Clustering por densidad (DBSCAN)

- Puede identificar clusters con formas **no esféricas** y **poco definidas**, permitiendo una potencial **superposición** entre ellos.
- Permite un clustering **parcial** (eliminando elementos que no pertenecen a ningún cluster).
- Sesgo por densidad: puede tener problemas para identificar clusters con **densidades muy distintas** (porque se elige un único *Eps*).

Clustering por densidad (DBSCAN)

Una success story de DBSCAN



Clustering por densidad (DBSCAN)

<https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html>

```
class sklearn.cluster.DBSCAN (eps=0.5, min_samples=5, metric='euclidean', metric_params=None,
algorithm='auto', leaf_size=30, p=None, n_jobs=None)
```

[\[source\]](#)

Perform DBSCAN clustering from vector array or distance matrix.

DBSCAN - Density-Based Spatial Clustering of Applications with Noise. Finds core samples of high density and expands clusters from them. Good for data which contains clusters of similar density.

Read more in the [User Guide](#).

Parameters: **eps : float, optional**

The maximum distance between two samples for one to be considered as in the neighborhood of the other. This is not a maximum bound on the distances of points within a cluster. This is the most important DBSCAN parameter to choose appropriately for your data set and distance function.

Eps

min_samples : int, optional

The number of samples (or total weight) in a neighborhood for a point to be considered as a core point. This includes the point itself.

MinPts

metric : string, or callable

The metric to use when calculating distance between instances in a feature array. If metric is a string or callable, it must be one of the options allowed by [sklearn.metrics.pairwise_distances](#) for its metric parameter. If metric is "precomputed", X is assumed to be a distance matrix and must be square. X may be a sparse matrix, in which case only "nonzero" elements may be considered neighbors for DBSCAN.

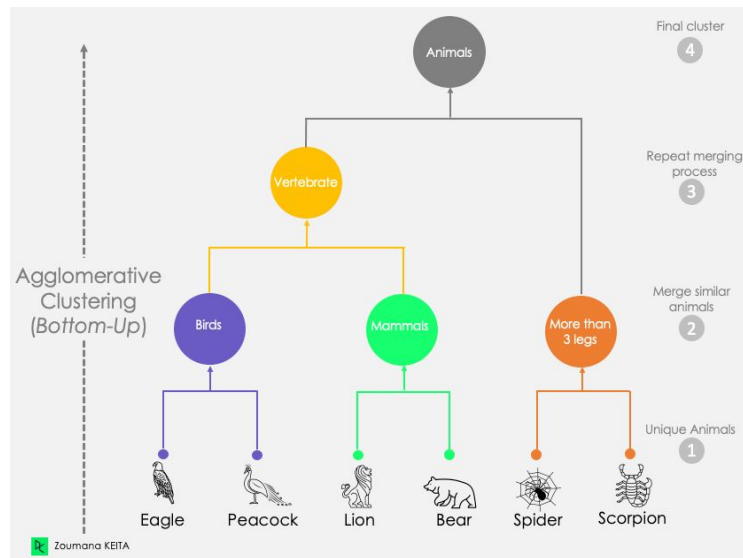
Se pueden usar varias medidas de similaridad, y se le puede pasar una precomputada.

New in version 0.17: metric precomputed to accept precomputed sparse matrix.

Clustering jerárquico

- Aglomerativo: Se parte de puntos individuales (*singleton*, hojas) y se van uniendo los más cercanos.
- Divisivo: Se parte de un sólo cluster (*root*, raíz) y se van separando los más disímiles hasta quedarse sólo con los puntos individuales.

Dendrograma
(árbol de jerarquía de similitudes)



Clustering jerárquico aglomerativo

0. Computar la matriz de similaridad.



Obs: hay que elegir medida de distancia (o similaridad) entre puntos, y cómo voy a proceder al comparar la similaridad entre clusters

Repetir:

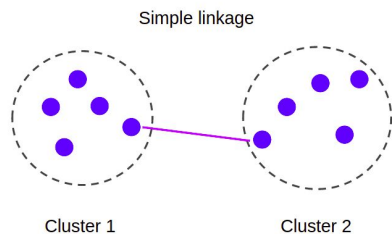
1. Juntar los dos más cercanos.

2. Actualizar la matriz de similaridad utilizando el nuevo cluster.

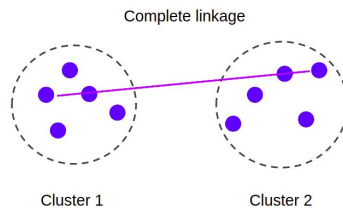
Hasta que: Sólo quede un cluster

Clustering jerárquico aglomerativo

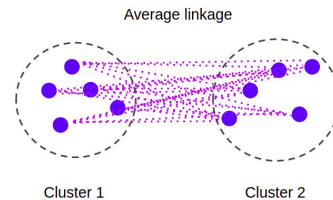
¿Cómo evaluamos la “cercanía” entre clusters? ¿Cómo elegimos el par de clusters a unir en cada paso?



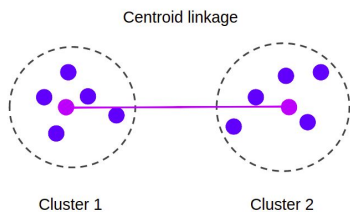
Mínima distancia entre puntos



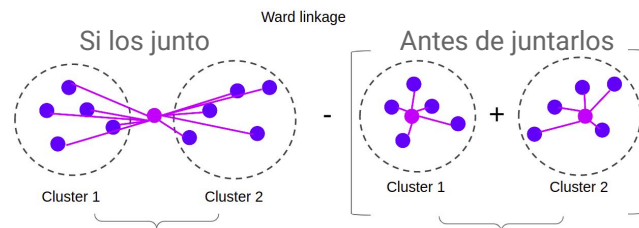
Máxima distancia entre puntos



Distancia promedio entre puntos



Distancia entre centroides



Función objetivo: Evalúa el “costo” (medido como la varianza intra-cluster) que resulte de juntar un par de clusters

Clustering jerárquico aglomerativo

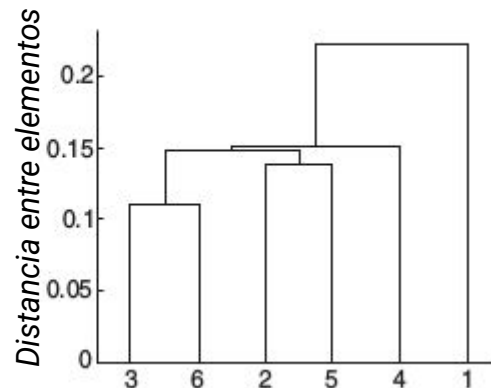
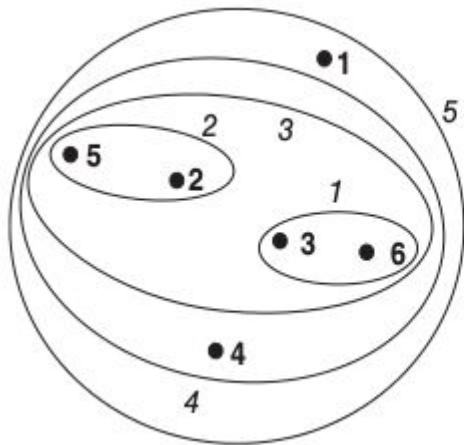
0. Computar la matriz de similaridad.

Repetir:

1. Juntar los dos más cercanos.
2. Actualizar la matriz de similaridad utilizando el nuevo cluster.

Hasta que: Sólo quede un cluster

Estructura jerarquizada
de distancias



Clustering jerárquico aglomerativo

0. Computar la matriz de similaridad.

Repetir:

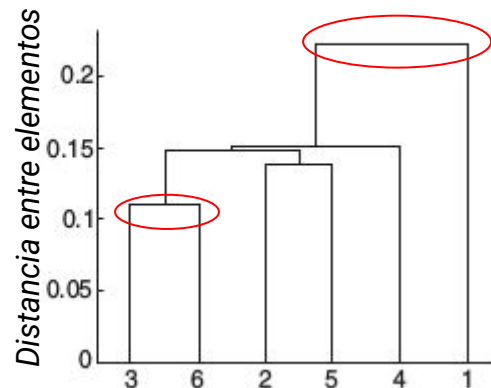
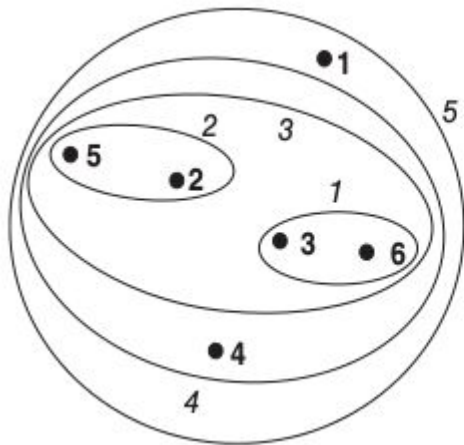
1. Juntar los dos más cercanos.
2. Actualizar la matriz de similaridad utilizando el nuevo cluster.

Hasta que: Sólo quede un cluster

Distancias “cofenéticas”

La distancia cofenética entre:

- El punto 1 y el cluster que engloba el resto de los puntos es de ~ 0.22
- Entre los puntos 3 y 6 es de ~ 0.11

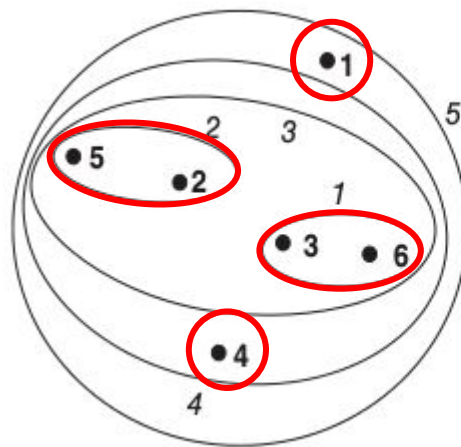
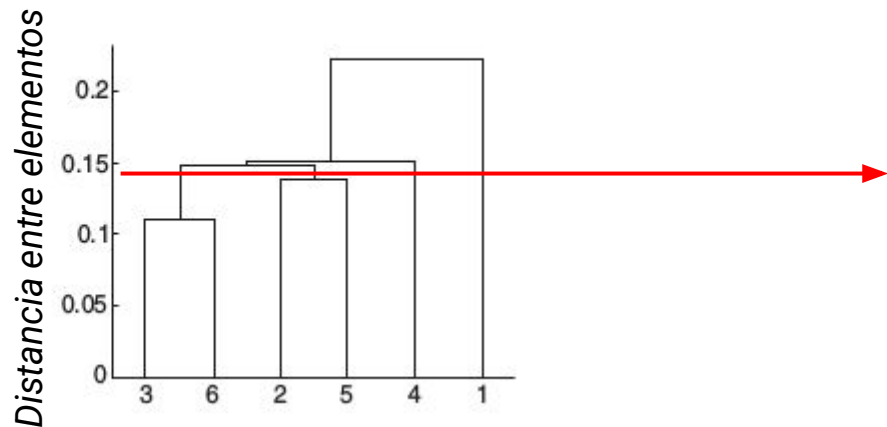


Clustering jerárquico aglomerativo

- Se pueden usar **nociones de proximidad arbitrarias**.
- **Minimiza propiedades locales, no globales** (ya que tomo decisiones en base a pares).
Por lo que la solución no necesariamente es el mínimo global. Muchas veces el resultado final se utiliza como inicialización de *k-means* u otro por partición, acomodando los resultados al mínimo global con una buena inicialización.
- Aporta una noción de **jerarquía** además de grupos.
- Para generar los grupos es necesario establecer un **criterio de corte**.
- **Complejidad**: $O(N^3)$ es peor para datasets grandes.

Clustering jerárquico aglomerativo

- Para generar los grupos es necesario establecer un criterio de corte.

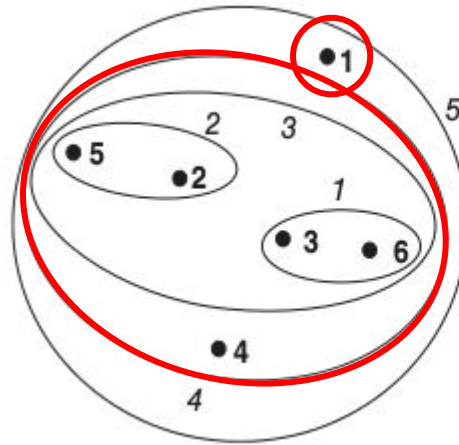
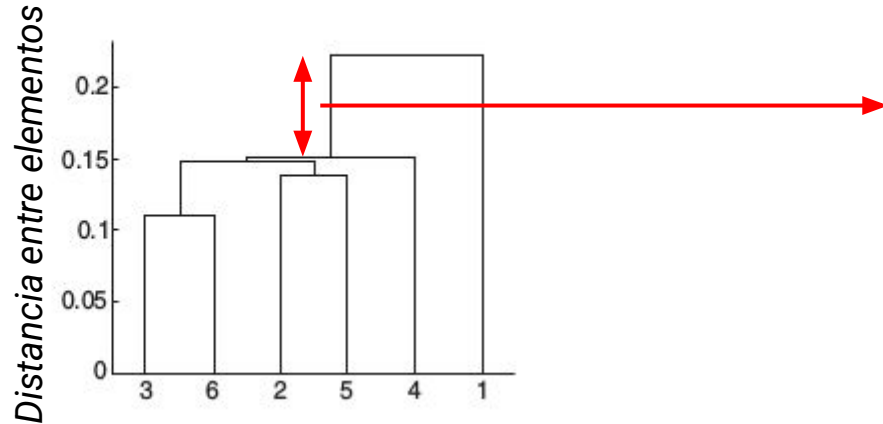


Clustering jerárquico aglomerativo

- Para generar los grupos es necesario establecer un criterio de corte.

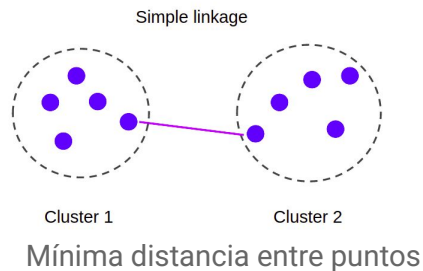
Un criterio posible: cortar justo **antes del salto más grande** en el dendrograma (evitando incluir una unión que resulte en mucha dispersión)

Vamos a ver más detalles de métodos para validar los cortes en la clase que viene

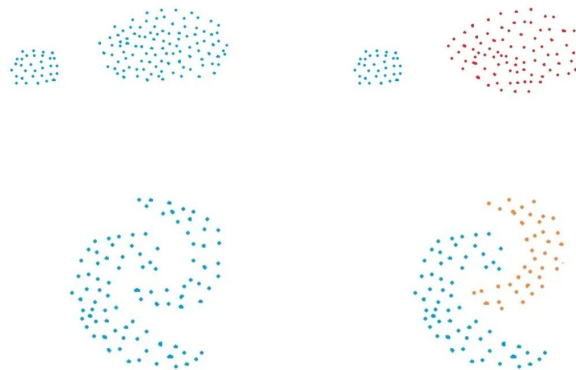


Clustering jerárquico aglomerativo

Comparemos los linkages



Pro: puede trabajar bien con clusters no elípticos

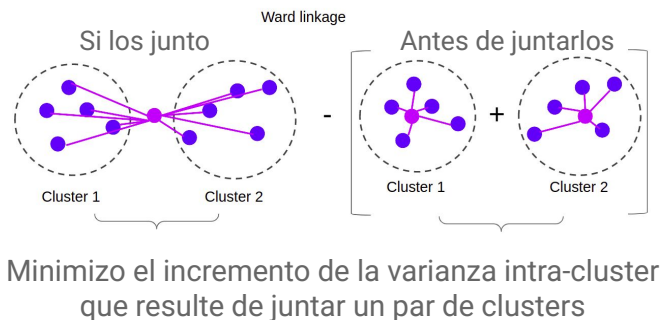
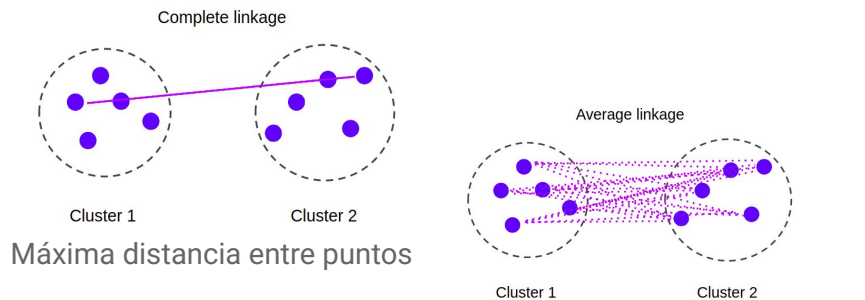


Contra: susceptible al “ruido” entre clusters



Clustering jerárquico aglomerativo

Comparemos los linkages



Pro: pueden resolver situaciones de “ruido” entre clusters



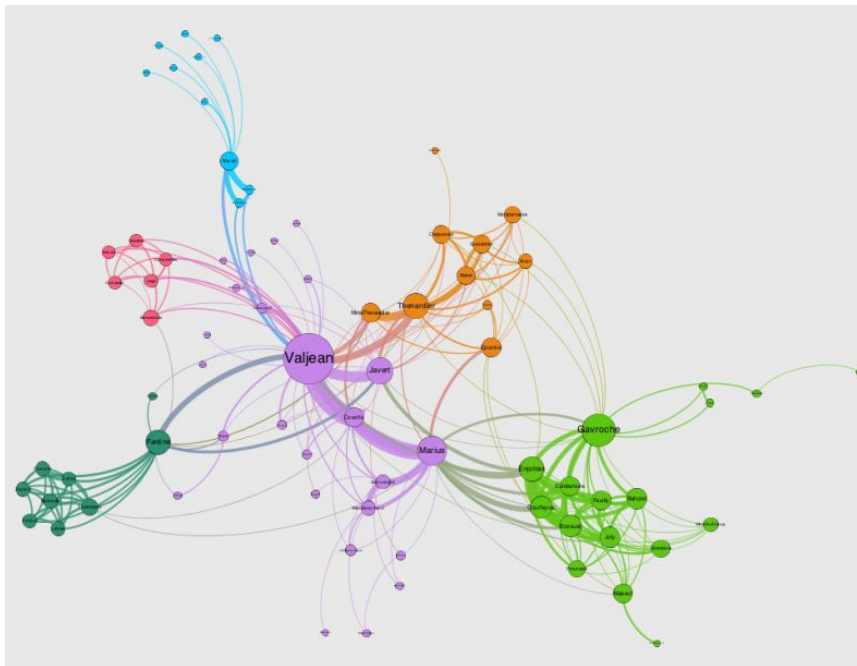
Contra:

- tienen un sesgo hacia clusters globulares
- Complete linkage tiende a “romper” clusters grandes



Clustering jerárquico aglomerativo

Una *success story* de clustering jerárquico



Clustering jerárquico aglomerativo

<https://scikit-learn.org/stable/modules/generated/sklearn.cluster.AgglomerativeClustering.html>

```
class sklearn.cluster. AgglomerativeClustering (n_clusters=2, affinity='euclidean', memory=None, connectivity=None, compute_full_tree='auto', linkage='ward', pooling_func='deprecated', distance_threshold=None)
```

[\[source\]](#)

Agglomerative Clustering

Recursively merges the pair of clusters that minimally increases a given linkage distance.

Read more in the [User Guide](#).

Parameters: **n_clusters** : *int or None, optional (default=2)*

The number of clusters to find. It must be `None` if `distance_threshold` is not `None`.

affinity : *string or callable, default: "euclidean"*

Metric used to compute the linkage. Can be "euclidean", "l1", "l2", "manhattan", "cosine", or "precomputed". If linkage is "ward", only "euclidean" is accepted. If "precomputed", a distance matrix (instead of a similarity matrix) is needed as input for the fit method.

Se pueden usar varias medidas de similaridad, y se le puede pasar una precomputada.

Clustering difuso o c-means

<https://pythonhosted.org/scikit-fuzzy/>

No se asigna cada elemento exclusivamente a un cluster, si no que existe una **probabilidad de pertenecer a cada uno de los clusters**.

Pesos (Función) de membresía,

$$w_{ij}(x_i) \in [0, 1] \quad \begin{array}{l} i=1\dots m \text{ puntos} \\ j=1\dots k \text{ clusters} \end{array}$$

Para cada elemento x_i

Sumando sobre clusters $\longrightarrow \sum_{j=1}^k w_{ij} = 1$

Para cada cluster C_j

$$0 < \sum_{i=1}^m w_{ij} < m$$

Sumando sobre puntos $\longrightarrow$

Clustering difuso o c-means

El parámetro p mide cuán rápido decae la pertenencia en función de la distancia (cuanto mayor sea, más rápido decae)

Algoritmo.

0. Elijo el número de clusters k

1. Inicializo w_{ij}

2. **Repite**

→ 3. Calculo/Actualizo los centroides c_k
a partir de los pesos w_{ij} y los puntos x_i

4. Calculo/Actualizo w_{ij}

5. **hasta** que los centroides no cambien o $SSE < \epsilon$
si no, vuelve a 1.

$$c_j = \frac{\sum_{i=1}^m w_{ij}^p x_i}{\sum_{i=1}^m w_{ij}^p}$$

El cálculo de los centroides es un promedio pesado

Clustering difuso o c-means

El parámetro p mide cuán rápido decae la pertenencia en función de la distancia (cuanto mayor sea, más rápido decae)

Algoritmo.

0. Elijo el número de clusters k
1. Inicializo w_{ij}
2. **Repite**
 3. Calculo/Actualizo los centroides c_k
a partir de los pesos w_{ij} y los puntos x_i
 - ➡ 4. Calculo/Actualizo w_{ij}
5. **hasta** que los centroides no cambien o $SSE < \epsilon$
si no, vuelve a 1.

$$c_j = \frac{\sum_{i=1}^m w_{ij}^p x_i}{\sum_{i=1}^m w_{ij}^p}$$

$$w_{ij} = \frac{(1/\text{dist}(x_i, c_j)^2)^{\frac{1}{p-1}}}{\sum_{q=1}^k (1/\text{dist}(x_i, c_q)^2)^{\frac{1}{p-1}}}$$

Actualización análoga a k-means: basada en la distancia del punto al centroide

Clustering difuso o c-means

Algoritmo.

0. Elijo el número de clusters k

1. Inicializo w_{ij}

2. **Repite**

3. Calculo/Actualizo los centroides c_k
a partir de los pesos w_{ij} y los puntos x_i

4. Calculo/Actualizo w_{ij}

5. **hasta** que los centroides no cambien o $SSE < \epsilon$
➡ **si no**, vuelve a 1.

$$c_j = \frac{\sum_{i=1}^m w_{ij}^p x_i}{\sum_{i=1}^m w_{ij}^p}$$

$$w_{ij} = \frac{(1/\text{dist}(x_i, c_j)^2)^{\frac{1}{p-1}}}{\sum_{q=1}^k (1/\text{dist}(x_i, c_q)^2)^{\frac{1}{p-1}}}$$

$$SSE = \sum_{j=1}^k \sum_{i=1}^m w_{ij}^p \text{dist}(x_i, c_j)^2$$

Clustering difuso o c-means

Algoritmo.

0. Elijo el número de clusters k
1. Inicializo w_{ij}
2. **Repite**
 3. Calculo/Actualizo los centroides c_k
a partir de los pesos w_{ij} y los puntos x_i
 4. Calculo/Actualizo w_{ij}
5. **hasta** que los centroides no cambien o $SSE < \epsilon$
si no, vuelve a 1.

Si así lo quisiera, aplicando un umbral (w_{th}) sobre w_{ij} puedo hacer asignación “duras” a un cluster, o incluso considerar un punto “ruido” (por ej., si $w_{ij} < w_{th}$ para todo j)

$p=2$

$$c_j = \frac{\sum_{i=1}^m w_{ij}^p x_i}{\sum_{i=1}^m w_{ij}^p}$$

$$w_{ij} = \frac{(1/\text{dist}(x_i, c_j)^2)^{\frac{1}{p-1}}}{\sum_{q=1}^k (1/\text{dist}(x_i, c_q)^2)^{\frac{1}{p-1}}}$$

$$SSE = \sum_{j=1}^k \sum_{i=1}^m w_{ij}^p \text{dist}(x_i, c_j)^2$$

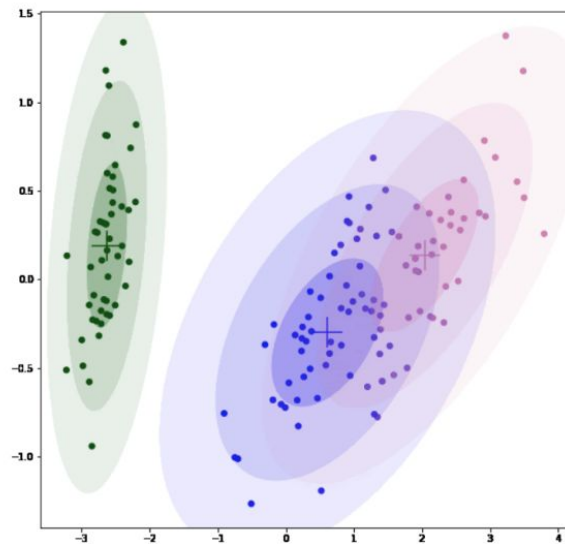
Clustering difuso o c-means

<https://pythonhosted.org/scikit-fuzzy/>

- **Cada elemento no es asignado exclusivamente a un cluster**, si no que existe una probabilidad de pertenecer a cada uno de los clusters.
- Permite asignar varios clusters a un elemento (p.ej. cuando un individuo tiene distintos roles en una organización, o un gen participa de distintas funciones), también permite separar objetos en una imagen con bordes difusos (no asignarlos a ningún cluster).
- Tiene desventajas similares a k-means, además de que es mucho más demandante computacionalmente.

Clustering difuso o c-means

Una success story de clustering difuso



Resumen

Escenarios óptimos para cada método

k-means

Es muy escaleable, siendo capaz de manejar grandes conjuntos de datos.

Particularmente eficaz cuando hay una comprensión o estimación preliminar de en cuántos grupos se debe segmentar el conjunto de datos.

Los conjuntos de datos que inherentemente tienen una forma de cúmulo globular son candidatos ideales.

Agrupamiento difuso (c-means)

Similar a k-means, pero con una extensión “probabilística” que permite asignar a cada punto una “confianza” de asignación a un cluster. También permite hacer un agrupamiento superpuesto (múltiples pertenencias para cada punto) o parcial (eliminando puntos). Pero computacionalmente mucho más pesado.

Similares sesgos hacia cúmulos globulares.

Resumen

Escenarios óptimos para cada método

Agrupación jerárquica

Brilla en el análisis de datos exploratorios. Comprender la estructura jerárquica inherente de los datos puede ofrecer información importante que otras técnicas de agrupación podrían pasar por alto.

Más adecuada para conjuntos de datos más pequeños.

Si el análisis exige múltiples niveles de granularidad de agrupación y una variedad de niveles de segmentación, la agrupación jerárquica es una buena elección.

Dependiendo del linkage, puede trabajar con agrupamientos no globulares (min linkage), con un costo de susceptibilidad mayor al ruido.

Resumen

Escenarios óptimos para cada método

DBSCAN

Sobresale cuando el conjunto de datos contiene ruido significativo o puntos de datos irrelevantes.

Si sus datos muestran grupos no globulares o de forma irregular, DBSCAN suele ser una opción más adecuada que los métodos basados en centroides. Al basarse en la densidad, le permite contornear los datos de forma natural.

Los conjuntos de datos espaciales, donde la proximidad y la densidad de los puntos de datos tienen importancia, se alinean bien con las capacidades de DBSCAN. Por ej., agrupar datos geoespaciales, o segmentar objetos en una imagen ruidosa.

Próxima clase

- Medidas de Distancias / Proximidad / Similitud / Disimilaridad / ...
- Métodos de validación interna (no supervisado)
- Métodos de validación externa (supervisado)